SAKTHI: Sparse Tucker Acceleration via Adaptive Kernels for TTMc and SVD in HOOI

Marati Bhaskar Dr. Raghavendra Kanakagiri

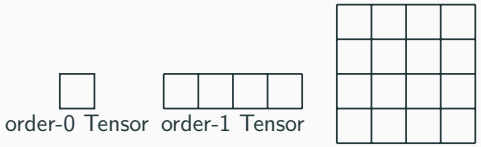
ACM Transactions on Architecture and Code Optimization (TACO), 2026

The Presentation in five steps

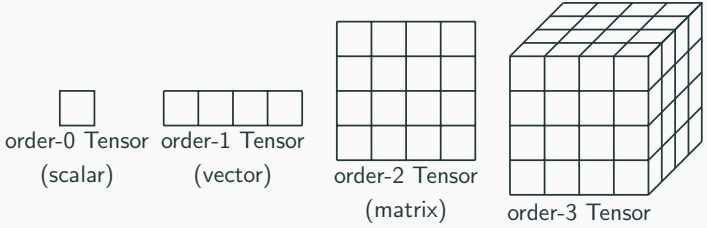
How do we find hidden patterns in huge, mostly empty, multi-dimensional data, quickly?



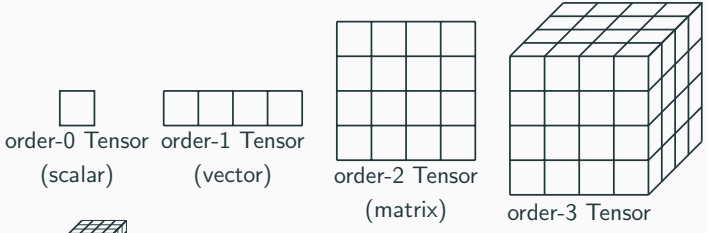
Tensor Visualization



Tensor Visualization



Tensor Visualization



Introduction

- Tensors are multi-dimensional arrays with applications in:
  - Quantum computing
  - Biomedical data analysis
  - Pattern recognition
  - Signal processing
  - Recommender systems
- **Tensor Decomposition (TD)** factorizes tensors into simpler components
- Two most popular TD methods:
  - CANDECOMP/PARAFAC (CP) decomposition
  - Tucker decomposition

Motivation: Application of Tensor Decomposition

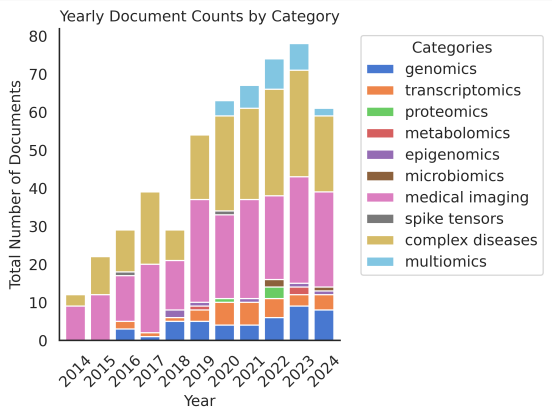
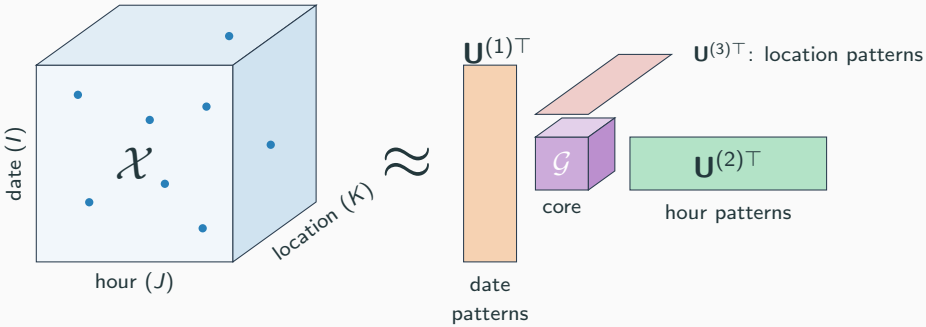


Figure 1: Tensor decomposition applications in biomedical data over the past decade (PubMed data, Burch et al. 2025)

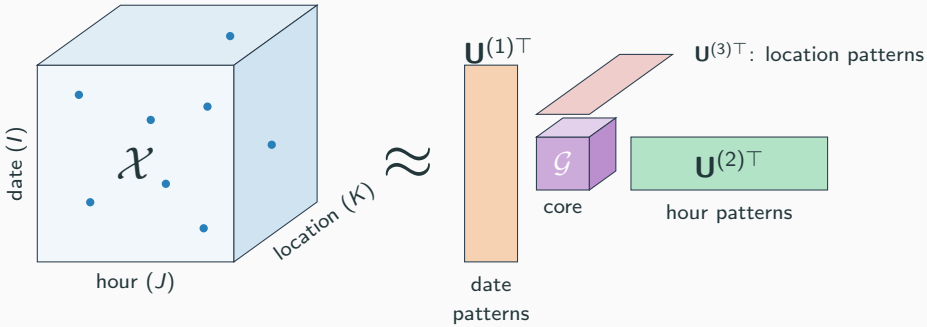
SAKTHI: Sparse Tucker Acceleration via Adaptive Kernels for TTMc and SVD in HOOI

Tucker decomposition



A small **core** $\mathcal{G}$ plus one thin **factor matrix** per dimension.
The ranks R_n (e.g. 10) are tiny next to the dimensions I, J, K (up to millions).

Tucker decomposition

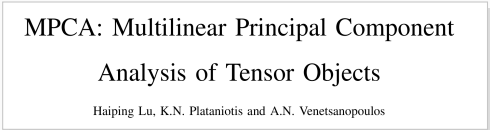


A small **core** $\mathcal{G}$ plus one thin **factor matrix** per dimension.

The ranks R_n (e.g. 10) are tiny next to the dimensions I, J, K (up to millions).

Each factor holds the main patterns of its dimension. The core says how they combine.

Tucker decomposition is used across fields

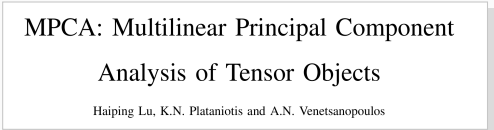

MPCA: Multilinear Principal Component Analysis of Tensor Objects

Haiping Lu, K.N. Plataniotis and A.N. Venetsanopoulos

Face and gait recognition

Lu et al., IEEE Trans. Neural Networks 2008

Tucker decomposition is used across fields


MPCA: Multilinear Principal Component Analysis of Tensor Objects

Haiping Lu, K.N. Plataniotis and A.N. Venetsanopoulos

Face and gait recognition

Lu et al., IEEE Trans. Neural Networks 2008


SIAM REVIEW
Vol. 51, No. 3, pp. 455–500

© 2009 Society for Industrial and Applied Mathematics

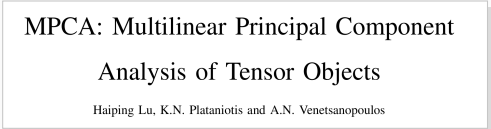
Tensor Decompositions and Applications*

Tamara G. Kolda†
Brett W. Bader‡

The standard survey: Tucker as higher-order PCA

Kolda & Bader, SIAM Review 2009

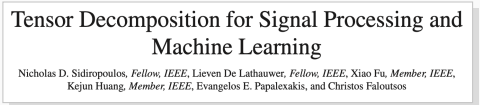
Tucker decomposition is used across fields


MPCA: Multilinear Principal Component Analysis of Tensor Objects

Haiping Lu, K.N. Plataniotis and A.N. Venetsanopoulos

Face and gait recognition

Lu et al., IEEE Trans. Neural Networks 2008


Tensor Decomposition for Signal Processing and Machine Learning

Nicholas D. Sidiropoulos, *Fellow, IEEE*, Lieven De Lathauwer, *Fellow, IEEE*, Xiao Fu, *Member, IEEE*, Kejun Huang, *Member, IEEE*, Evangelos E. Papalexakis, and Christos Faloutsos

Signal processing and machine learning

Sidiropoulos et al., IEEE TSP 2017


SIAM REVIEW
Vol. 51, No. 3, pp. 455–500

© 2009 Society for Industrial and Applied Mathematics

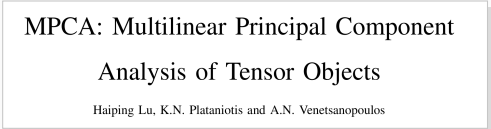
Tensor Decompositions and Applications*

Tamara G. Kolda†
Brett W. Bader‡

The standard survey: Tucker as higher-order PCA

Kolda & Bader, SIAM Review 2009

Tucker decomposition is used across fields


MPCA: Multilinear Principal Component Analysis of Tensor Objects

Haiping Lu, K.N. Plataniotis and A.N. Venetsanopoulos

Face and gait recognition

Lu et al., IEEE Trans. Neural Networks 2008


SIAM REVIEW
Vol. 51, No. 3, pp. 455–500

© 2009 Society for Industrial and Applied Mathematics

Tensor Decompositions and Applications*

Tamara G. Kolda†
Brett W. Bader‡

The standard survey: Tucker as higher-order PCA

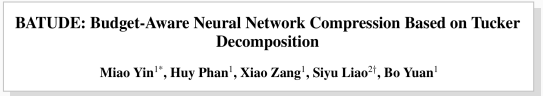
Kolda & Bader, SIAM Review 2009


Tensor Decomposition for Signal Processing and Machine Learning

Nicholas D. Sidiropoulos, *Fellow, IEEE*, Lieven De Lathauwer, *Fellow, IEEE*, Xiao Fu, *Member, IEEE*, Kejun Huang, *Member, IEEE*, Evangelos E. Papalexakis, and Christos Faloutsos

Signal processing and machine learning

Sidiropoulos et al., IEEE TSP 2017

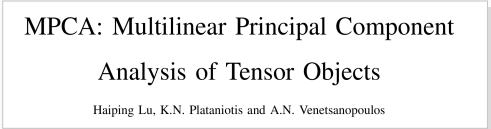

BATUDE: Budget-Aware Neural Network Compression Based on Tucker Decomposition

Miao Yin[1*], Huy Phan[1], Xiao Zang[1], Siyu Liao[2†], Bo Yuan[1]

Compressing deep neural networks

Yin et al., AAAI 2022

Tucker decomposition is used across fields


MPCA: Multilinear Principal Component Analysis of Tensor Objects

Haiping Lu, K.N. Plataniotis and A.N. Venetsanopoulos

Face and gait recognition

Lu et al., IEEE Trans. Neural Networks 2008


SIAM REVIEW
Vol. 51, No. 3, pp. 455–500

© 2009 Society for Industrial and Applied Mathematics

Tensor Decompositions and Applications*

Tamara G. Kolda†
Brett W. Bader‡

The standard survey: Tucker as higher-order PCA

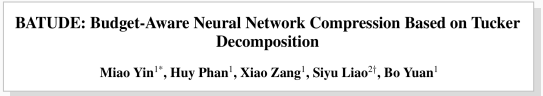
Kolda & Bader, SIAM Review 2009


Tensor Decomposition for Signal Processing and Machine Learning

Nicholas D. Sidiropoulos, *Fellow, IEEE*, Lieven De Lathauwer, *Fellow, IEEE*, Xiao Fu, *Member, IEEE*, Kejun Huang, *Member, IEEE*, Evangelos E. Papalexakis, and Christos Faloutsos

Signal processing and machine learning

Sidiropoulos et al., IEEE TSP 2017


BATUDE: Budget-Aware Neural Network Compression Based on Tucker Decomposition

Miao Yin[1*], Huy Phan[1], Xiao Zang[1], Siyu Liao[2†], Bo Yuan[1]

Compressing deep neural networks

Yin et al., AAAI 2022

Tucker finds hidden structure in data across many fields

SAKTHI: Sparse Tucker Acceleration via Adaptive Kernels for TTMc and SVD in HOOI

Higher-Order Orthogonal Iteration (HOOI)

Alternating Least Squares (ALS) algorithm to compute rank-$(R_1, R_2, \ldots, R_N)$ Tucker decomposition for an N-th order tensor $\mathcal{X}$ of size $I_1 \times I_2 \times \cdots \times I_N$

Algorithm 1 Higher-Order Orthogonal Iteration (HOOI)[1]

Input: Tensor $\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times \cdots \times I_N}$ and target ranks $(R_1, R_2, \ldots, R_N)$.

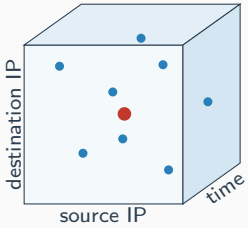
1: **for** $n = 1$ **to** N **do**
2: Initialize $\mathbf{A}^{(n)} \leftarrow R_n$ leading left singular vectors of $\mathbf{X}_{(n)}$
3: **end for**
4: **while** fit has not converged **and** max iterations not reached **do**
5: **for** $n = 1$ **to** N **do**
6: $\mathcal{Y} \leftarrow \mathcal{X} \times_1 \mathbf{A}^{(1)\top} \cdots \times_{n-1} \mathbf{A}^{(n-1)\top} \times_{n+1} \mathbf{A}^{(n+1)\top} \cdots \times_N \mathbf{A}^{(N)\top}$ ▷ *TTMc*
7: $\mathbf{A}^{(n)} \leftarrow R_n$ leading left singular vectors of $\mathcal{Y}_{(n)}$ ▷ *SVD*
8: **end for**
9: **end while**
10: $\mathcal{G} \leftarrow \mathcal{X} \times_1 \mathbf{A}^{(1)\top} \times_2 \mathbf{A}^{(2)\top} \cdots \times_N \mathbf{A}^{(N)\top}$
11: **return** $\mathcal{G}, \mathbf{A}^{(1)}, \mathbf{A}^{(2)}, \ldots, \mathbf{A}^{(N)}$

[1] Kolda and Bader 2009.

SAKTHI: Sparse Tucker Acceleration via Adaptive Kernels for TTMc and SVD in HOOI

Sparse data

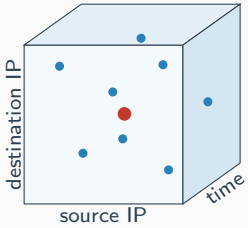
Network traffic



a cell: traffic from one IP to another at one time

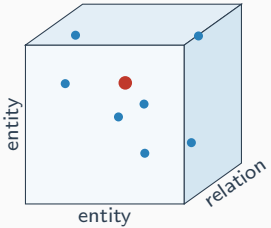
Sparse data

Network traffic



a cell: traffic from one IP to another at one time

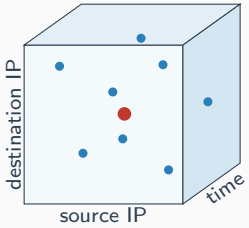
Knowledge base



a cell: 1 if a fact is known, e.g. (Tirupati, located in, India)

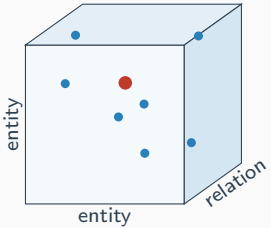
Sparse data

Network traffic



a cell: traffic from one IP to another at one time

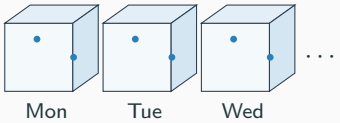
Knowledge base



a cell: 1 if a fact is known, e.g. (Tirupati, located in, India)

Uber pickups (4D)

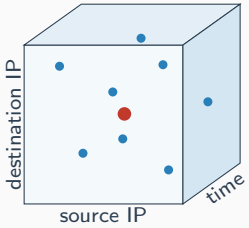
hour × latitude × longitude, per date



a cell: pickups at one place in one hour of one day

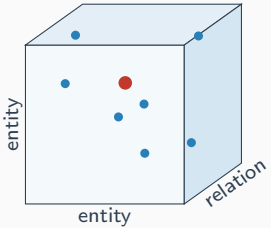
Sparse data

Network traffic



a cell: traffic from one IP to another at one time

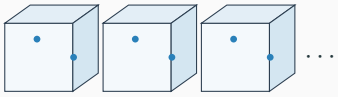
Knowledge base



a cell: 1 if a fact is known, e.g. (Tirupati, located in, India)

Uber pickups (4D)

hour × latitude × longitude, per date



a cell: pickups at one place in one hour of one day

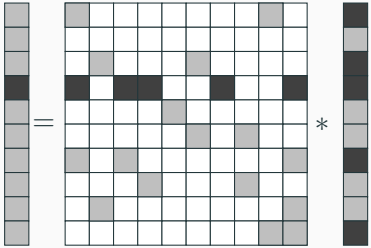
Most cells are empty: most IPs never talk to each other, most facts are unknown. Such data is **sparse**. A multi-dimensional array is called a **tensor**.

Challenge: dense algorithms waste work on zeros

- Real-world data is sparse, e.g. a users $\times$ movies rating matrix

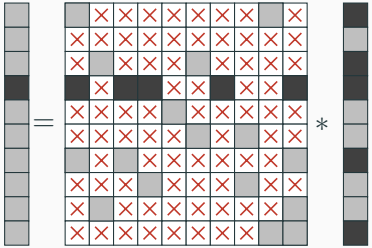
Challenge: dense algorithms waste work on zeros

- Real-world data is sparse, e.g. a users $\times$ movies rating matrix



Challenge: dense algorithms waste work on zeros

- Real-world data is sparse, e.g. a users × movies rating matrix

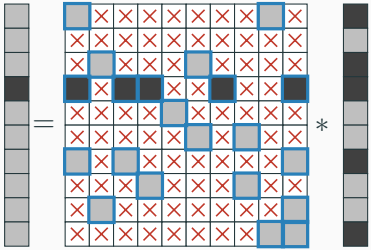


dense: 100 multiply-adds, 79 by zero

```
1 for (i = 0; i < I; i++) {
2   for (j = 0; j < J; j++) {
3       y[i] += A[i][j] * x[j];
4   }
5 } // O( I · J)
```

Challenge: dense algorithms waste work on zeros

- Real-world data is sparse, e.g. a users × movies rating matrix



dense: 100 multiply-adds, 79 by zero

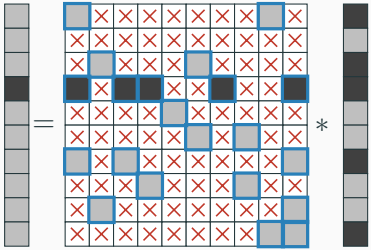
sparse: only the 21 nonzeros

```
for (i = 0; i < I; i++) {
  for (j = 0; j < J; j++) {
      y[i] += A[i][j] * x[j];
  }
} // O( I · J)
```

```
for (n = 0; n < nnz; n++) {
  i = indI(n), j = indJ(n);
  y[i] += A[i][j] * x[j];
} // O(nnz)
```

Challenge: dense algorithms waste work on zeros

- Real-world data is sparse, e.g. a users × movies rating matrix



dense: 100 multiply-adds, 79 by zero

sparse: only the 21 nonzeros

```
for (i = 0; i < I; i++) {
  for (j = 0; j < J; j++) {
      y[i] += A[i][j] * x[j];
  }
} // O( I · J)
```

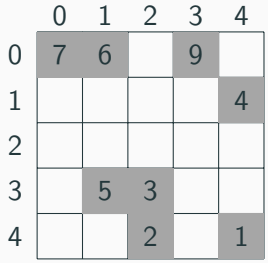
```
for (n = 0; n < nnz; n++) {
  i = indI(n), j = indJ(n);
  y[i] += A[i][j] * x[j];
} // O(nnz)
```

Storing zeros also wastes memory: `nell1` (a knowledge base)

$2.9\text{M} \times 2.1\text{M} \times 25\text{M} \approx 1.5 \times 10^{20}$ cells, only 1.4×10^{8} nonzeros (1 in a trillion).

Dense storage ≈ **600 exabytes**. Sparse storage ≈ **4 GB**.

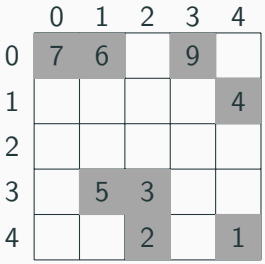
Sparse Storage Formats



Sparse Matrix

$O(mn)$

Sparse Storage Formats



Sparse Matrix

$O(mn)$

row_idx	col_idx	Value
0	0	7
0	1	6
0	3	9
1	4	4
3	1	5
3	2	3
4	2	2
4	4	1

Table 1:
Coordinate(COO) format

$O(3\,nnz)$

Sparse Storage Formats

	0	1	2	3	4
0	7	6		9	
1					4
2					
3		5	3		
4			2		1

Sparse Matrix

$O(mn)$

row_idx	col_idx	Value
0	0	7
0	1	6
0	3	9
1	4	4
3	1	5
3	2	3
4	2	2
4	4	1

Table 1:
Coordinate(COO) format

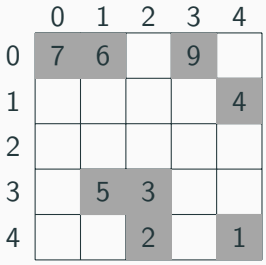
$O(3\,nnz)$

0	1	2	3	4	5	6	7	
0	1	3	4	1	2	2	4	col_idx
7	6	9	4	5	3	2	1	val

CSR format

$O(2\,nnz + m + 1)$

Sparse Storage Formats



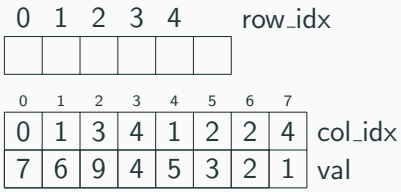
Sparse Matrix

$O(mn)$

row_idx	col_idx	Value
0	0	7
0	1	6
0	3	9
1	4	4
3	1	5
3	2	3
4	2	2
4	4	1

Table 1:
Coordinate(COO) format

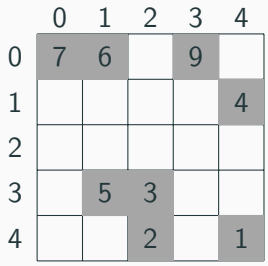
$O(3\,nnz)$



CSR format

$O(2\,nnz + m + 1)$

Sparse Storage Formats



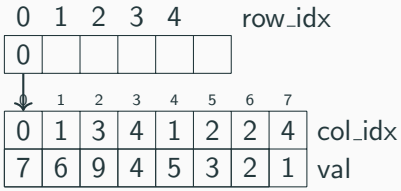
Sparse Matrix

$O(mn)$

row_idx	col_idx	Value
0	0	7
0	1	6
0	3	9
1	4	4
3	1	5
3	2	3
4	2	2
4	4	1

Table 1:
Coordinate(COO) format

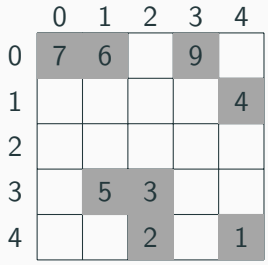
$O(3\,nnz)$



CSR format

$O(2\,nnz + m + 1)$

Sparse Storage Formats



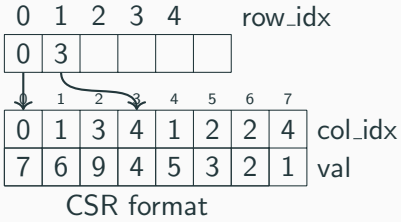
Sparse Matrix

$O(mn)$

row_idx	col_idx	Value
0	0	7
0	1	6
0	3	9
1	4	4
3	1	5
3	2	3
4	2	2
4	4	1

Table 1:
Coordinate(COO) format

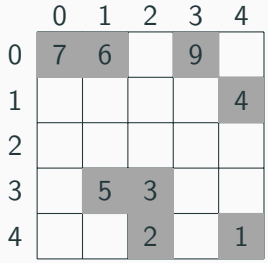
$O(3\,nnz)$



CSR format

$O(2\,nnz + m + 1)$

Sparse Storage Formats



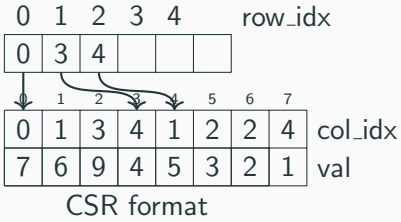
Sparse Matrix

$O(mn)$

row_idx	col_idx	Value
0	0	7
0	1	6
0	3	9
1	4	4
3	1	5
3	2	3
4	2	2
4	4	1

Table 1:
Coordinate(COO) format

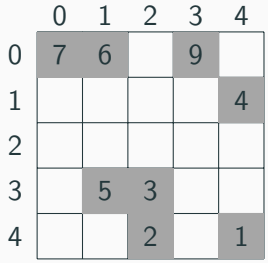
$O(3\,nnz)$



CSR format

$O(2\,nnz + m + 1)$

Sparse Storage Formats



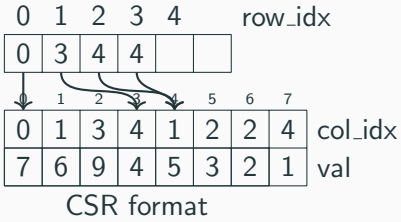
Sparse Matrix

$O(mn)$

row_idx	col_idx	Value
0	0	7
0	1	6
0	3	9
1	4	4
3	1	5
3	2	3
4	2	2
4	4	1

Table 1:
Coordinate(COO) format

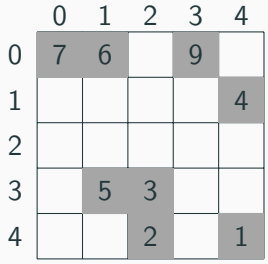
$O(3\,nnz)$



CSR format

$O(2\,nnz + m + 1)$

Sparse Storage Formats



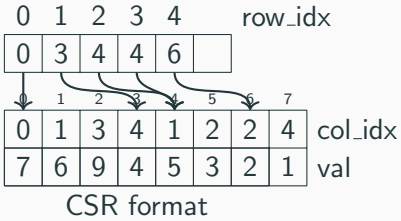
Sparse Matrix

$O(mn)$

row_idx	col_idx	Value
0	0	7
0	1	6
0	3	9
1	4	4
3	1	5
3	2	3
4	2	2
4	4	1

Table 1:
Coordinate(COO) format

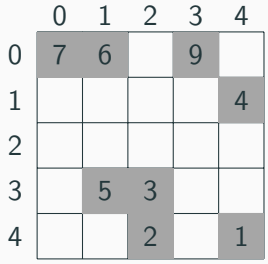
$O(3\,nnz)$



CSR format

$O(2\,nnz + m + 1)$

Sparse Storage Formats



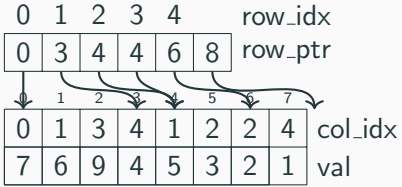
Sparse Matrix

$O(mn)$

row_idx	col_idx	Value
0	0	7
0	1	6
0	3	9
1	4	4
3	1	5
3	2	3
4	2	2
4	4	1

Table 1:
Coordinate(COO) format

$O(3\,nnz)$



CSR format

$O(2\,nnz + m + 1)$

Sparse Storage Formats

	0	1	2	3	4
0	7	6		9	
1					4
2					
3		5	3		
4			2		1

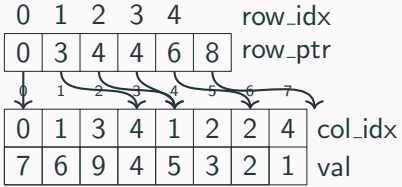
Sparse Matrix

$O(mn)$

row_idx	col_idx	Value
0	0	7
0	1	6
0	3	9
1	4	4
3	1	5
3	2	3
4	2	2
4	4	1

Table 2:
Coordinate(COO) format

$O(3\,nnz)$

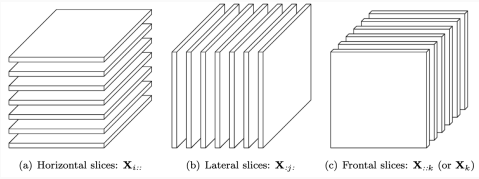


CSR format

$O(2\,nnz + m + 1)$

```
1 for (i = 0; i < I; i++) {
2   for (j = r_p[i]; j < r_p[i+1]; j++) {
3     y[i] += val[j] * x[col_idx[j]];
4   } //O(nnz)
5 }
```

Running example: slices, fibers and nonzeros



Slices (left) and fibers (right) of a 3D tensor, from Kolda and Bader 2009

A $2 \times 3 \times 4$ tensor $\mathcal{X}$:

24 cells, only 6 nonzeros.

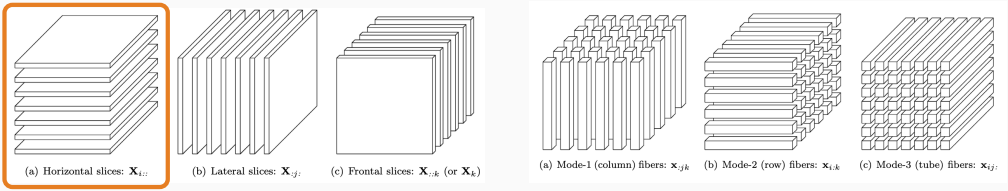
slice $i = 0$

	k=0	k=1	k=2	k=3
j=0	a			
j=1		b	c	
j=2	d			e

slice $i = 1$

	k=0	k=1	k=2	k=3
j=0				
j=1	f			
j=2				

Running example: slices, fibers and nonzeros



Slices (left) and fibers (right) of a 3D tensor, from Kolda and Bader 2009

A $2 \times 3 \times 4$ tensor $\mathcal{X}$:

24 cells, only 6 nonzeros.

- **Slice**: fix i, a 2D sheet

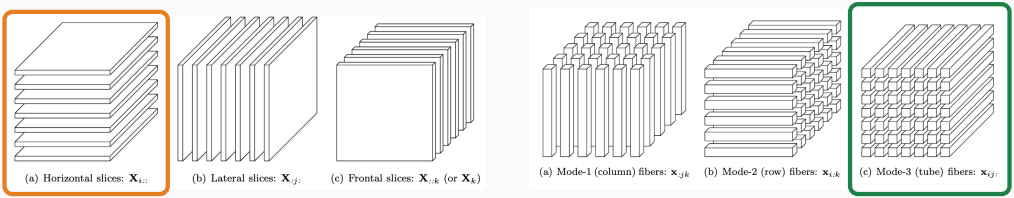
slice $i = 0$

	k=0	k=1	k=2	k=3
j=0	a			
j=1		b	c	
j=2	d			e

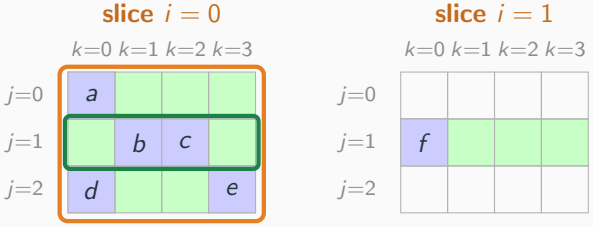
slice $i = 1$

	k=0	k=1	k=2	k=3
j=0				
j=1	f			
j=2				

Running example: slices, fibers and nonzeros



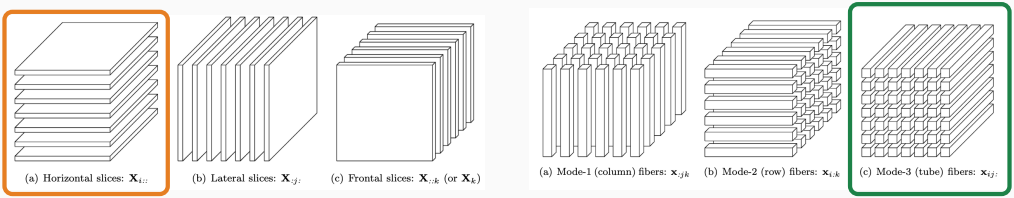
Slices (left) and fibers (right) of a 3D tensor, from Kolda and Bader 2009



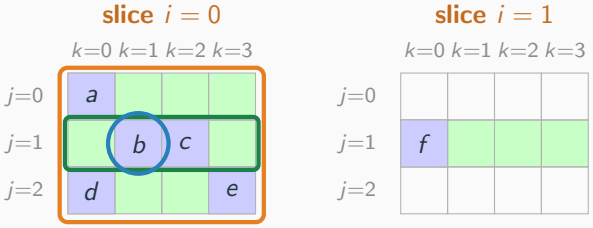
A $2 \times 3 \times 4$ tensor $\mathcal{X}$:
24 cells, only 6 nonzeros.

- **Slice**: fix i, a 2D sheet
- **Fiber**: fix i and j, one row of a sheet (4 non-empty fibers here)

Running example: slices, fibers and nonzeros



Slices (left) and fibers (right) of a 3D tensor, from Kolda and Bader 2009



A $2 \times 3 \times 4$ tensor $\mathcal{X}$:
24 cells, only 6 nonzeros.

- **Slice**: fix i, a 2D sheet
- **Fiber**: fix i and j, one row of a sheet (4 non-empty fibers here)
- **Nonzero**: one stored value, e.g. $x_{0,1,1} = b$

CSF: storing a sparse tensor as a tree

COO

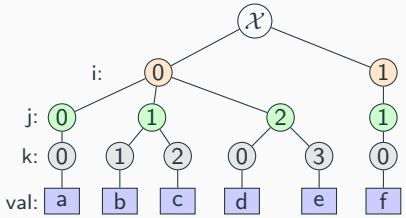
i	j	k	val
0	0	0	a
0	1	1	b
0	1	2	c
0	2	0	d
0	2	3	e
1	1	0	f

CSF: storing a sparse tensor as a tree

COO

i	j	k	val
0	0	0	a
0	1	1	b
0	1	2	c
0	2	0	d
0	2	3	e
1	1	0	f

CSF tree



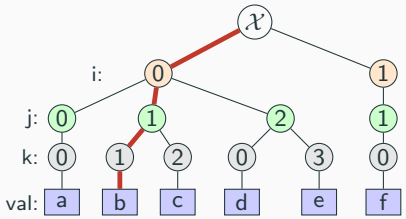
slice index i fiber index j nonzero index k value

CSF: storing a sparse tensor as a tree

COO

i	j	k	val
0	0	0	a
0	1	1	b
0	1	2	c
0	2	0	d
0	2	3	e
1	1	0	f

CSF tree



slice index i fiber index j nonzero index k value

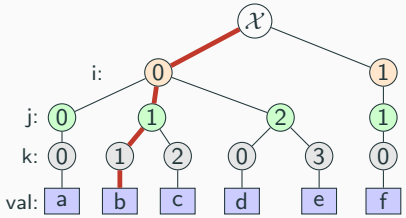
Each COO row is one root-to-leaf path, e.g. $(0, 1, 1, b)$.

CSF: storing a sparse tensor as a tree

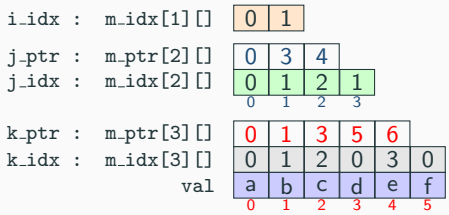
COO

i	j	k	val
0	0	0	a
0	1	1	b
0	1	2	c
0	2	0	d
0	2	3	e
1	1	0	f

CSF tree



CSF arrays



slice index i fiber index j nonzero index k value

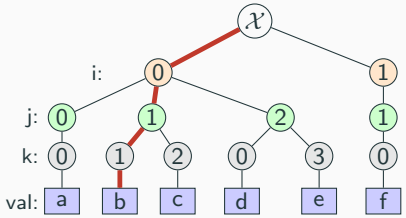
Each COO row is one root-to-leaf path, e.g. $(0, 1, 1, b)$.

CSF: storing a sparse tensor as a tree

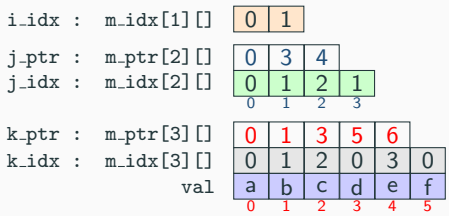
COO

i	j	k	val
0	0	0	a
0	1	1	b
0	1	2	c
0	2	0	d
0	2	3	e
1	1	0	f

CSF tree



CSF arrays



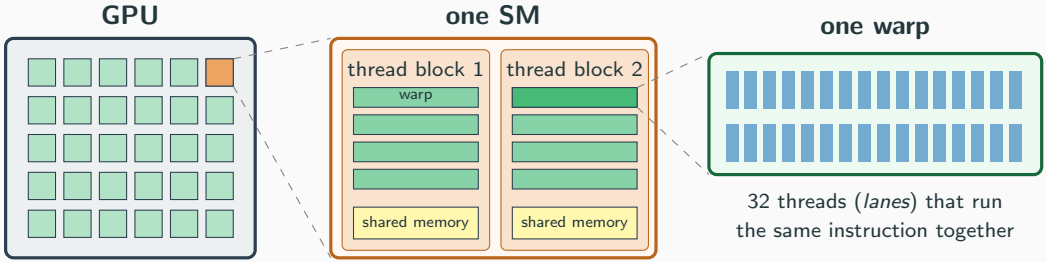
slice index i fiber index j nonzero index k value

Each COO row is one root-to-leaf path, e.g. $(0, 1, 1, b)$.

✔ a fiber's nonzeros sit together (one subtree) ✔ shared index prefixes are stored once

SAKTHI: Sparse Tucker Acceleration via Adaptive Kernels for TTMc and SVD in HOOI

A GPU: thousands of workers, organized in teams

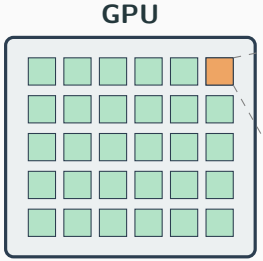


142 **SMs** (streaming multiprocessors) on an L40S

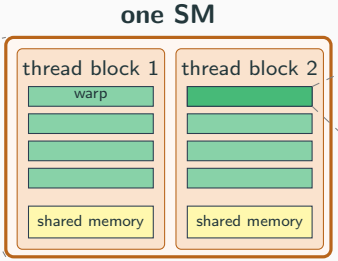
thread blocks of warps, each block with fast shared memory

32 threads (*lanes*) that run the same instruction together

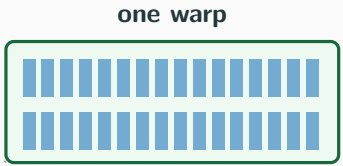
A GPU: thousands of workers, organized in teams



142 **SMs** (streaming multiprocessors) on an L40S



thread blocks of warps, each block with fast shared memory



32 threads (*lanes*) that run the same instruction together



Keep everyone busy

the slowest worker decides when the job is done



Read neighbouring data

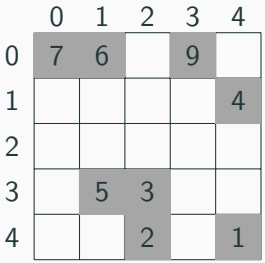
32 lanes reading 32 neighbours need one memory trip



Avoid write conflicts

writes to one location take turns (atomics), slow across blocks

Sparse Storage Formats



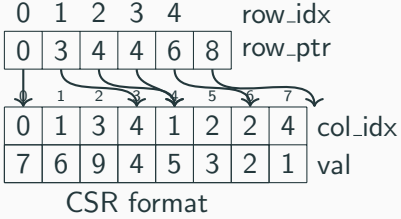
Sparse Matrix

$O(mn)$

row_idx	col_idx	Value
0	0	7
0	1	6
0	3	9
1	4	4
3	1	5
3	2	3
4	2	2
4	4	1

Table 3:
Coordinate(COO) format

$O(3\,nnz)$

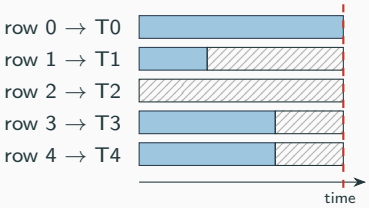


CSR format

$O(2\,nnz + m + 1)$

Why parallel sparse computation is hard

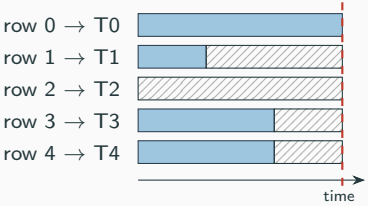
1 Uneven work



The busiest worker decides when all are done. Real data: 1 to 42,390 nonzeros per fiber (`nell1`).

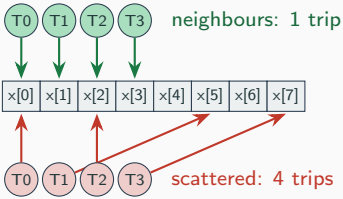
Why parallel sparse computation is hard

1 Uneven work



The busiest worker decides when all are done. Real data: 1 to 42,390 nonzeros per fiber (`nell1`).

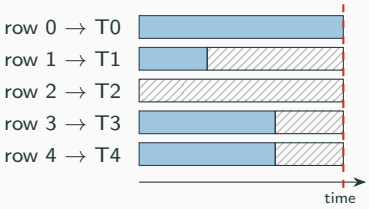
2 Scattered memory reads



`x[col_idx[j]]` jumps around memory, so a warp needs many slow trips.

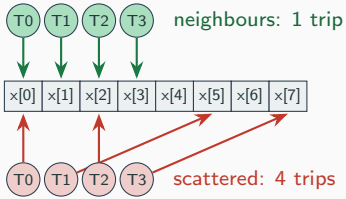
Why parallel sparse computation is hard

1 Uneven work



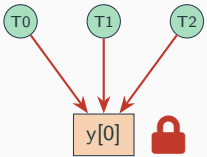
The busiest worker decides when all are done. Real data: 1 to 42,390 nonzeros per fiber (`nell1`).

2 Scattered memory reads



`x[col_idx[j]]` jumps around memory, so a warp needs many slow trips.

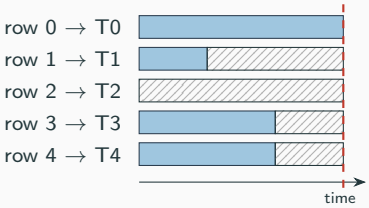
3 Write conflicts



Splitting a long row balances the work, but then all workers add into y[0] and must take turns (atomics).

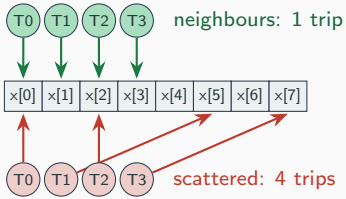
Why parallel sparse computation is hard

❶ Uneven work



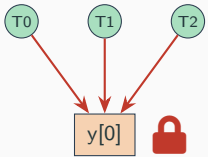
The busiest worker decides when all are done. Real data: 1 to 42,390 nonzeros per fiber (`nell1`).

❷ Scattered memory reads



`x[col_idx[j]]` jumps around memory, so a warp needs many slow trips.

❸ Write conflicts

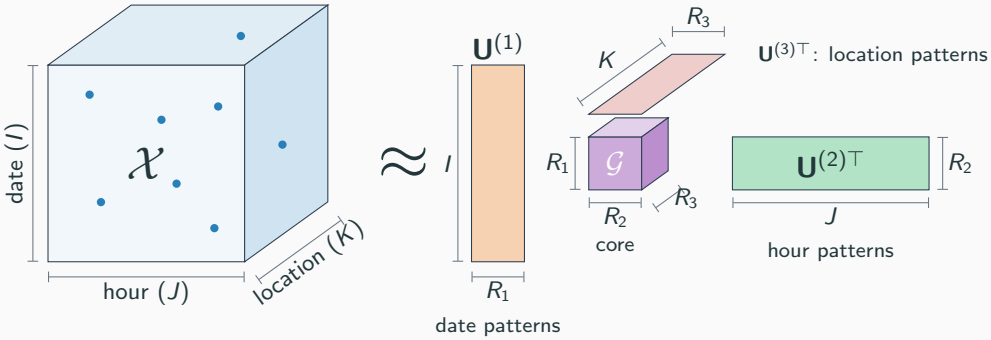


Splitting a long row balances the work, but then all workers add into y[0] and must take turns (atomics).

Balancing work, regular memory access and conflict-free writes pull against each other.

SAKTHI: Sparse Tucker Acceleration via Adaptive Kernels for TTMc and SVD in HOOI

Tucker decomposition



TTMc: the kernel we need to compute

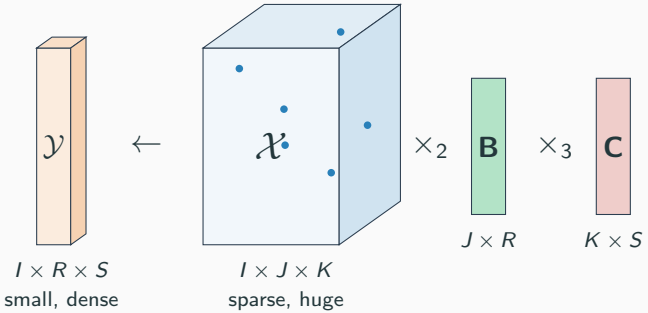
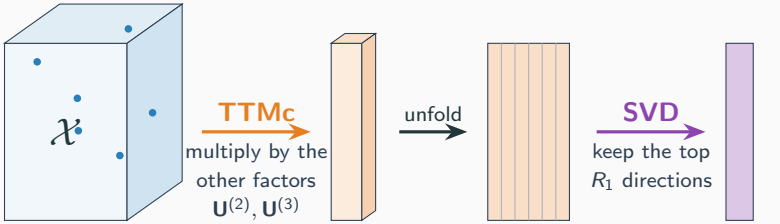


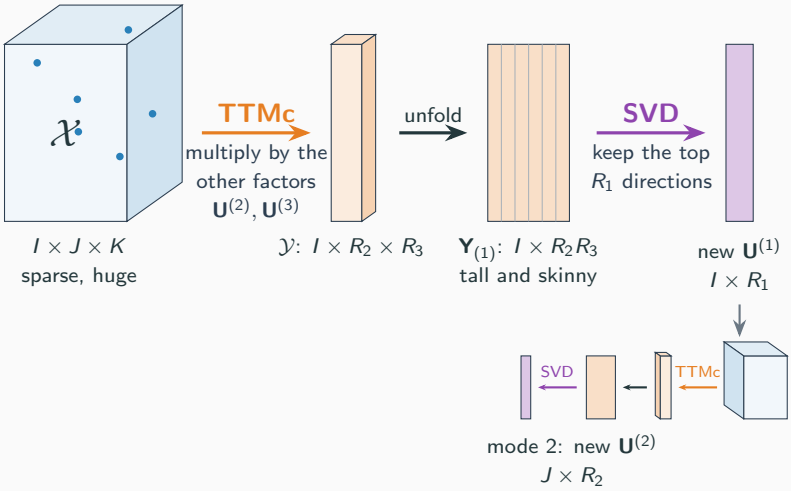
Figure 2: TTMc kernel ($\mathcal{Y}_{irs} \leftarrow \sum_{j,k} \mathcal{X}_{ijk} \mathbf{B}_{jr} \mathbf{C}_{ks}$) for order-3 Tensor

$\mathbf{B} = \mathbf{U}^{(2)}$ and $\mathbf{C} = \mathbf{U}^{(3)}$ are the factor matrices of the other two modes, so $R = R_2$ and $S = R_3$.

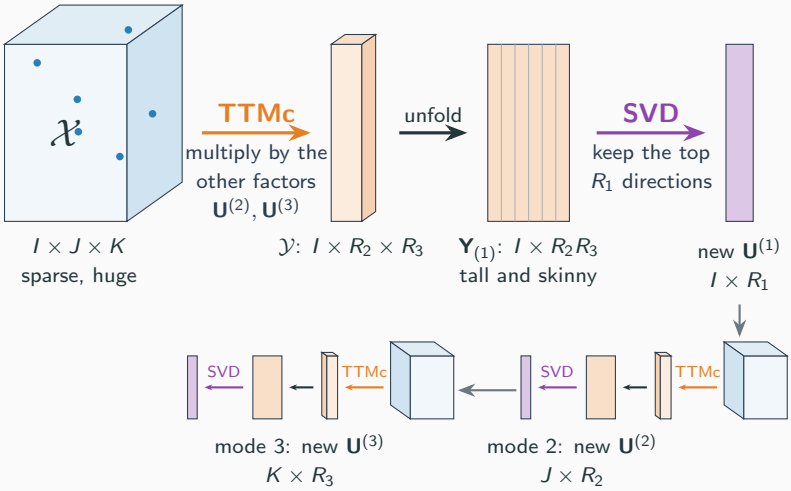
Computing Tucker: HOOI repeats two steps



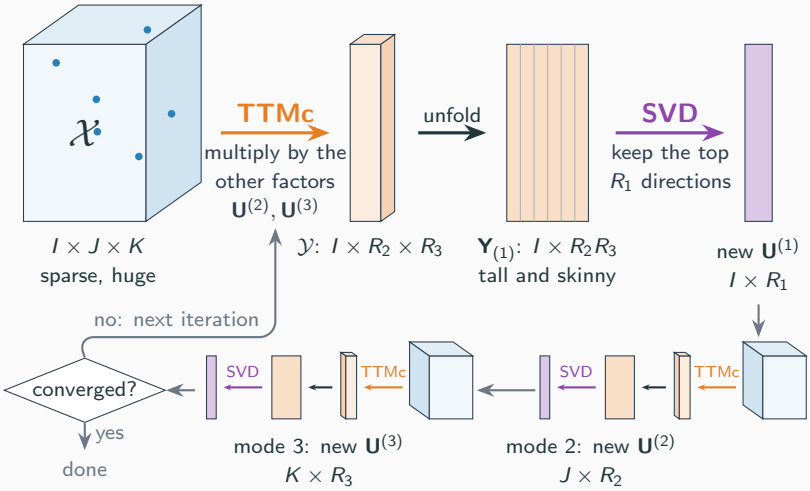
Computing Tucker: HOOI repeats two steps



Computing Tucker: HOOI repeats two steps



Computing Tucker: HOOI repeats two steps



Prior work speeds up TTM and TTMc

Optimizing sparse tensor times matrix on GPUs

Yuchen Ma [a], Jiajia Li [b,*], Xiaolong Wu [c], Chenggang Yan [a], Jimeng Sun [b], Richard Vuduc [b]

ParTI!: Ma et al., the baseline we compare against

Prior work speeds up TTM and TTMc

Optimizing sparse tensor times matrix on GPUs

Yuchen Ma [a], Jiajia Li [b,*], Xiaolong Wu [c], Chenggang Yan [a], Jimeng Sun [b], Richard Vuduc [b]

ParTI!: Ma et al., the baseline we compare against

A Unified Optimization Approach for Sparse Tensor Operations on GPUs

Bangtian Liu*, Chengyao Wen*, Anand D. Sarwate*, Maryam Mehri Dehnavi*

Prior work speeds up TTM and TTMc

Optimizing sparse tensor times matrix on GPUs

Yuchen Ma [a], Jiajia Li [b,*], Xiaolong Wu [c], Chenggang Yan [a], Jimeng Sun [b], Richard Vuduc [b]

ParTI!: Ma et al., the baseline we compare against

A Unified Optimization Approach for Sparse Tensor Operations on GPUs

Bangtian Liu*, Chengyao Wen*, Anand D. Sarwate*, Maryam Mehri Dehnavi*

A Novel Parallel Algorithm for Sparse Tensor Matrix Chain Multiplication via TCU-Acceleration

Haotian Wang, Wangdong Yang, Rong Hu, Renqiu Ouyang, Kenli Li, *Senior Member, IEEE*, and Keqin Li, *Fellow, IEEE*

Prior work speeds up TTM and TTMc

Optimizing sparse tensor times matrix on GPUs

Yuchen Ma [a], Jiajia Li [b,*], Xiaolong Wu [c], Chenggang Yan [a], Jimeng Sun [b], Richard Vuduc [b]

ParTI!: Ma et al., the baseline we compare against

A Unified Optimization Approach for Sparse Tensor Operations on GPUs

Bangtian Liu*, Chengyao Wen*, Anand D. Sarwate*, Maryam Mehri Dehnavi*

A Novel Parallel Algorithm for Sparse Tensor Matrix Chain Multiplication via TCU-Acceleration

Haotian Wang, Wangdong Yang, Rong Hu, Renqiu Ouyang, Kenli Li, *Senior Member, IEEE*, and Keqin Li, *Fellow, IEEE*

A Heterogeneous Parallel Computing Approach Optimizing SpTTM on CPU-GPU via GCN

HAOTIAN WANG, WANGDONG YANG, RENQIU OUYANG, RONG HU, and KENLI LI, College of Computer Science and Electronic Engineering, Hunan University, China
KEQIN LI, Department of Computer Science, State University of New York, USA and College of Computer Science and Electronic Engineering, Hunan University, China

Prior work speeds up TTM and TTMc

Optimizing sparse tensor times matrix on GPUs

Yuchen Ma [a], Jiajia Li [b,*], Xiaolong Wu [c], Chenggang Yan [a], Jimeng Sun [b], Richard Vuduc [b]

ParTI!: Ma et al., the baseline we compare against

A Unified Optimization Approach for Sparse Tensor Operations on GPUs

Bangtian Liu*, Chengyao Wen*, Anand D. Sarwate*, Maryam Mehri Dehnavi*

A Novel Parallel Algorithm for Sparse Tensor Matrix Chain Multiplication via TCU-Acceleration

Haotian Wang, Wangdong Yang, Rong Hu, Renqiu Ouyang, Kenli Li, *Senior Member, IEEE*, and Keqin Li, *Fellow, IEEE*

A Heterogeneous Parallel Computing Approach Optimizing SpTTM on CPU-GPU via GCN

HAOTIAN WANG, WANGDONG YANG, RENQIU OUYANG, RONG HU, and KENLI LI, College of Computer Science and Electronic Engineering, Hunan University, China
KEQIN LI, Department of Computer Science, State University of New York, USA and College of Computer Science and Electronic Engineering, Hunan University, China

Exploring Data Layout for Sparse Tensor Times Dense Matrix on GPUs

KHALID AHMAD, University of Utah, USA
CRIS CECKA and MICHAEL GARLAND, NVIDIA Corporation, USA
MARY HALL, University of Utah, USA

Prior work speeds up TTM and TTMc

Optimizing sparse tensor times matrix on GPUs

Yuchen Ma [a], Jiajia Li [b,*], Xiaolong Wu [c], Chenggang Yan [a], Jimeng Sun [b], Richard Vuduc [b]

ParTI!: Ma et al., the baseline we compare against

A Unified Optimization Approach for Sparse Tensor Operations on GPUs

Bangtian Liu*, Chengyao Wen*, Anand D. Sarwate*, Maryam Mehri Dehnavi*

A Novel Parallel Algorithm for Sparse Tensor Matrix Chain Multiplication via TCU-Acceleration

Haotian Wang, Wangdong Yang, Rong Hu, Renqiu Ouyang, Kenli Li, *Senior Member, IEEE*, and Keqin Li, *Fellow, IEEE*

A Heterogeneous Parallel Computing Approach Optimizing SpTTM on CPU-GPU via GCN

HAOTIAN WANG, WANGDONG YANG, RENQIU OUYANG, RONG HU, and KENLI LI, College of Computer Science and Electronic Engineering, Hunan University, China
KEQIN LI, Department of Computer Science, State University of New York, USA and College of Computer Science and Electronic Engineering, Hunan University, China

Exploring Data Layout for Sparse Tensor Times Dense Matrix on GPUs

KHALID AHMAD, University of Utah, USA
CRIS CECKA and MICHAEL GARLAND, NVIDIA Corporation, USA
MARY HALL, University of Utah, USA

All five speed up the **TTM / TTMc kernel**. None of them optimizes the **SVD**.

Our insight 1: for sparse tensors, the SVD is costly too

TTMc cost $\propto$ (number of nonzeros) $\times P$

SVD cost $\propto$ (dimension length) $\times P^2$

$P =$ product of the other ranks, e.g. $10 \times 10 = 100$.

Our insight 1: for sparse tensors, the SVD is costly too

TTMc cost $\propto$ (number of nonzeros) $\times P$

SVD cost $\propto$ (dimension length) $\times P^2$

P = product of the other ranks, e.g. $10 \times 10 = 100$.

Sparser data $\Rightarrow$ fewer nonzeros $\Rightarrow$ TTMc gets cheaper, but the SVD does not.

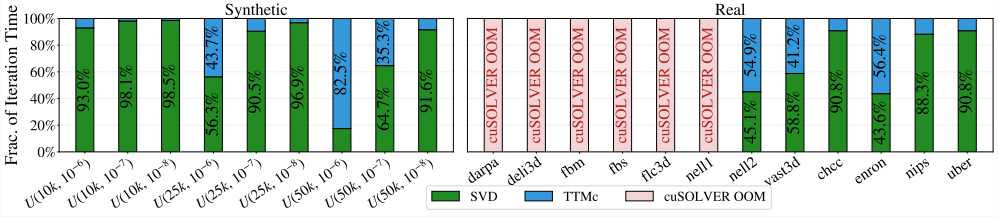
Our insight 1: for sparse tensors, the SVD is costly too

TTMc cost $\propto$ (number of nonzeros) $\times P$

SVD cost $\propto$ (dimension length) $\times P^2$

P = product of the other ranks, e.g. $10 \times 10 = 100$.

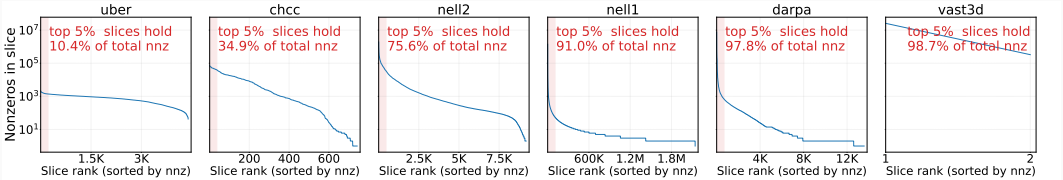
Sparser data $\Rightarrow$ fewer nonzeros $\Rightarrow$ TTMc gets cheaper, but the SVD does not.



Share of one HOOI iteration spent in TTMc and in full SVD (rank 10, GPU).

SVD takes up to 98.5% of the time. Full SVD runs out of memory on 6 of 12 real tensors.

Our insight 2: Real tensors are imbalanced, each in its own way



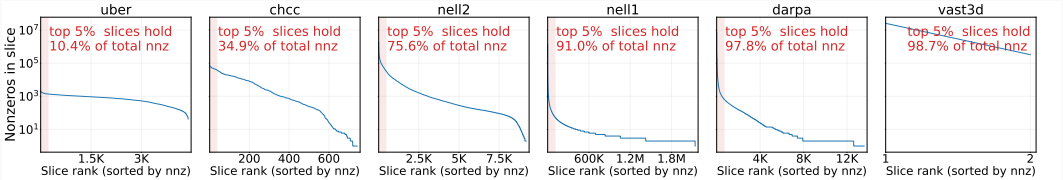
Nonzeros per slice, sorted from heaviest to lightest (log scale). Shaded: the heaviest 5% of slices.

`uber`: the top 5% of slices hold 10% of the nonzeros
$\Rightarrow$ fairly balanced

`nell1`: the top 5% hold 91%
$\Rightarrow$ a few giant slices

`vast3d`: only 2 slices
$\Rightarrow$ one block per slice uses 2 of 142 SMs

Our insight 2: Real tensors are imbalanced, each in its own way



Nonzeros per slice, sorted from heaviest to lightest (log scale). Shaded: the heaviest 5% of slices.

`uber`: the top 5% of slices hold 10% of the nonzeros
$\Rightarrow$ fairly balanced

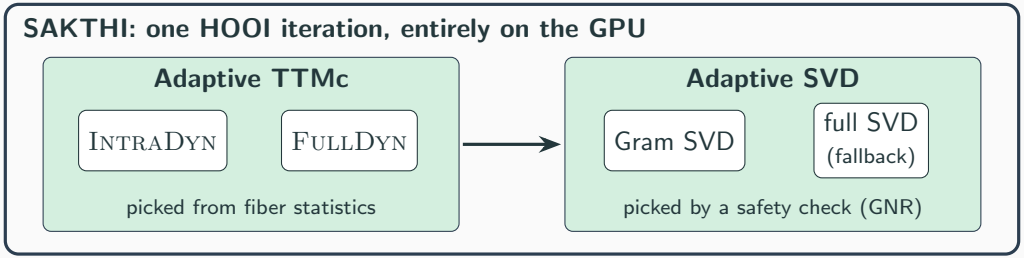
`nell1`: the top 5% hold 91%
$\Rightarrow$ a few giant slices

`vast3d`: only 2 slices
$\Rightarrow$ one block per slice uses 2 of 142 SMs

No single fixed way of splitting the work suits every tensor.

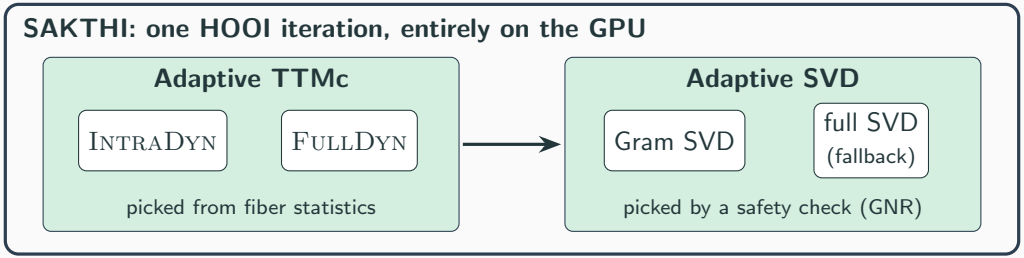
SAKTHI: Sparse Tucker Acceleration via Adaptive Kernels for TTMc and SVD in HOOI

SAKTHI at a glance



In contrast, ParTI! goes back to the CPU for index preparation and format conversions in every iteration.

SAKTHI at a glance



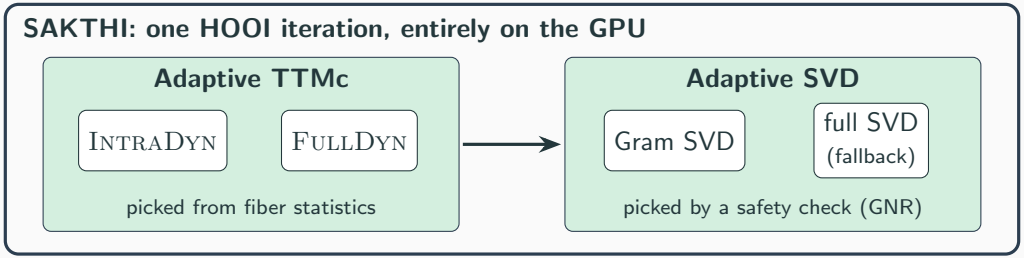
In contrast, ParTI! goes back to the CPU for index preparation and format conversions in every iteration.

1

Fuse, don't store

factorize-and-fuse on CSF:
no big intermediate tensor

SAKTHI at a glance



In contrast, ParTI! goes back to the CPU for index preparation and format conversions in every iteration.

1

Fuse, don't store

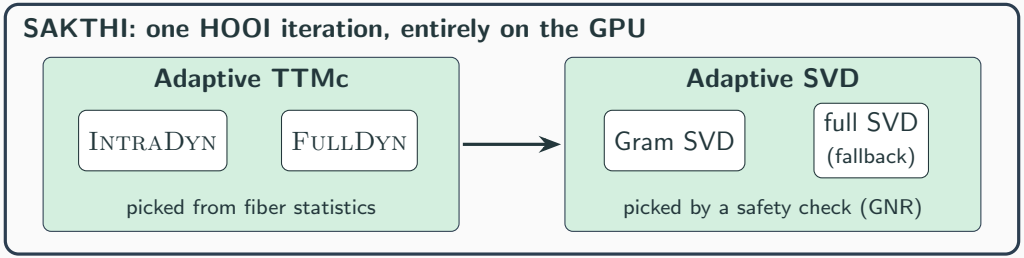
factorize-and-fuse on CSF:
no big intermediate tensor

2

Keep every warp busy

IntraDyn and FullDyn
dynamic scheduling

SAKTHI at a glance



In contrast, ParTI! goes back to the CPU for index preparation and format conversions in every iteration.

1. **Fuse, don't store**
   factorize-and-fuse on CSF:
   no big intermediate tensor

2. **Keep every warp busy**
   IntraDyn and FullDyn
   dynamic scheduling

3. **Shrink the SVD safely**
   Gram SVD with a
   computable safety check

Pairwise vs. factorize-and-fuse

TTMc: $y_{irs} = \sum_j \sum_k x_{ijk}\, \mathbf{B}_{jr}\, \mathbf{C}_{ks}$

Pairwise vs. factorize-and-fuse

TTMc: $y_{irs} = \sum_j \sum_k x_{ijk}\, \mathbf{B}_{jr}\, \mathbf{C}_{ks}$

Pairwise contraction (as in ParTI!): two separate steps

$$t_{ijs} = \sum_k x_{ijk}\, \mathbf{C}_{ks} \quad \text{then} \quad y_{irs} = \sum_j t_{ijs}\, \mathbf{B}_{jr}$$

The first step stores a dense intermediate $\mathcal{T} \in \mathbb{R}^{I \times J \times S}$, which can be prohibitively large.

Pairwise vs. factorize-and-fuse

TTMc: $y_{irs} = \sum_j \sum_k x_{ijk}\, \mathbf{B}_{jr}\, \mathbf{C}_{ks}$

Pairwise contraction (as in ParTI!): two separate steps

$$t_{ijs} = \sum_k x_{ijk}\, \mathbf{C}_{ks} \quad \text{then} \quad y_{irs} = \sum_j t_{ijs}\, \mathbf{B}_{jr}$$

The first step stores a dense intermediate $\mathcal{T} \in \mathbb{R}^{I \times J \times S}$, which can be prohibitively large.

Factorize-and-fuse (SAKTHI): the inner sum over k does not depend on r

$$y_{irs} = \sum_j \mathbf{B}_{jr} \underbrace{\left(\sum_k x_{ijk}\, \mathbf{C}_{ks} \right)}_{\mathtt{T}[s]} \qquad \mathbf{Y}_{i,:,:} = \sum_j \mathbf{B}_{j,:} \circ \underbrace{\left(\sum_k x_{ijk}\, \mathbf{C}_{k,:} \right)}_{\mathtt{T}[:]}$$

T is computed and used up within each fiber (i, j), so storage drops from $O(I \cdot J \cdot S)$ to $O(S)$.

Idea 1: fuse, don't store (factorize-and-fuse)

$ijk, ks \to ijs \to irs$

Listing 1: TTMc via pairwise contractions.

```
X_csf = CSF(X_ijk)
T = zeros(I,J,S)
for (i, X_i) in X_csf:
  for (j, X_ij) in X_i:
    for (k, x_ijk) in X_ij:
      for s in range(S):
        T[i,j,s] += x_ijk * C[k,s]
for (i, X_i) in X_csf:
  for (j, X_ij) in X_i:
    for s in range(S):
      for r in range(R):
        Y[i,r,s] += T[i,j,s] * B[j,r]
```

$ijk, jr, ks \to irs$

Listing 2: TTMc via factorize-and-fuse.

```
X_csf = CSF(X_ijk)
for (i, X_i) in X_csf:
  for (j, X_ij) in X_i:
    T = zeros(S) // local buffer
    for (k, x_ijk) in X_ij:
      for s in range(S):
        T[s] += x_ijk * C[k,s]
    for s in range(S):
      for r in range(R):
        Y[i,r,s] += T[s] * B[j,r]
```

Idea 1: fuse, don't store (factorize-and-fuse)

$ijk, ks \to ijs \to irs$

Listing 3: TTMc via pairwise contractions.

```
X_csf = CSF(X_ijk)
T = zeros(I,J,S)
for (i, X_i) in X_csf:
  for (j, X_ij) in X_i:
    for (k, x_ijk) in X_ij:
      for s in range(S):
        T[i,j,s] += x_ijk * C[k,s]
for (i, X_i) in X_csf:
  for (j, X_ij) in X_i:
    for s in range(S):
      for r in range(R):
        Y[i,r,s] += T[i,j,s] * B[j,r]
```

Pairwise (as in ParTI!) stores all of `T`, up to $I \times J \times S$ numbers

$ijk, jr, ks \to irs$

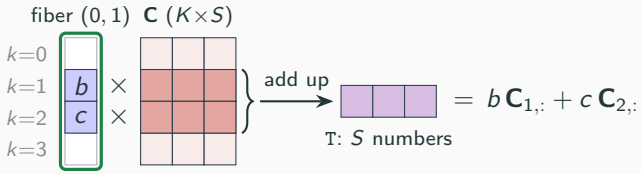
Listing 4: TTMc via factorize-and-fuse.

```
X_csf = CSF(X_ijk)
for (i, X_i) in X_csf:
  for (j, X_ij) in X_i:
    T = zeros(S) // local buffer
    for (k, x_ijk) in X_ij:
      for s in range(S):
        T[s] += x_ijk * C[k,s]
    for s in range(S):
      for r in range(R):
        Y[i,r,s] += T[s] * B[j,r]
```

Fused: `T` is a buffer of only S numbers, reused for every fiber

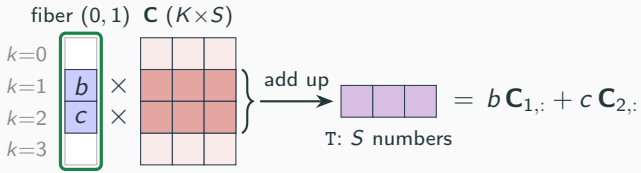
Factorize-and-fuse, one fiber at a time

1 Fill T from the fiber's nonzeros

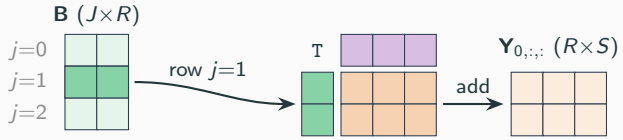


Factorize-and-fuse, one fiber at a time

1 Fill T from the fiber's nonzeros

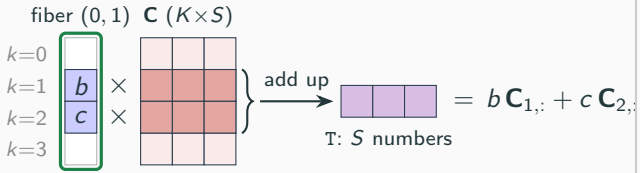


2 Drain T into the output slice

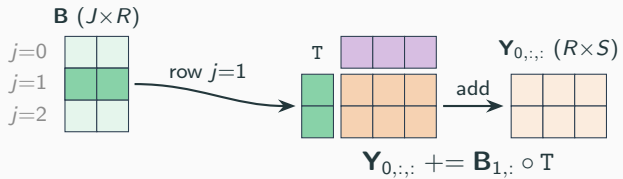


Factorize-and-fuse, one fiber at a time

1 Fill T from the fiber's nonzeros



2 Drain T into the output slice



3 Next fiber: reuse T

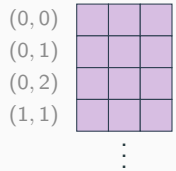
SAKTHI (fused)



cleared and reused for the next fiber

only S numbers at any time

Pairwise (ParTI!)

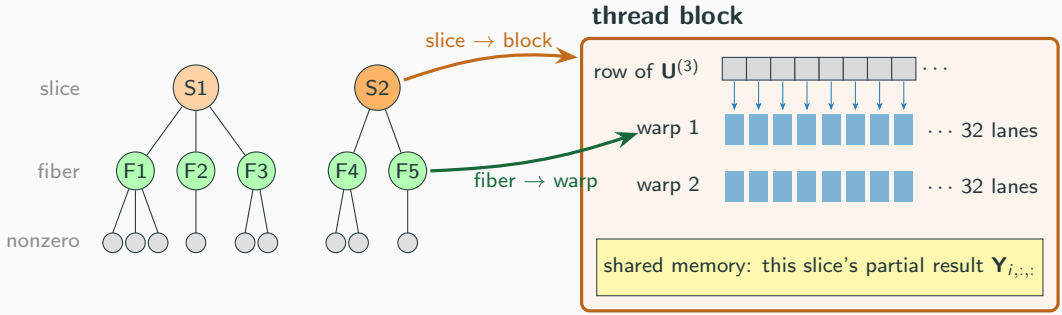


one T per fiber, all kept until step 2 runs

up to $I\times J\times S$ numbers

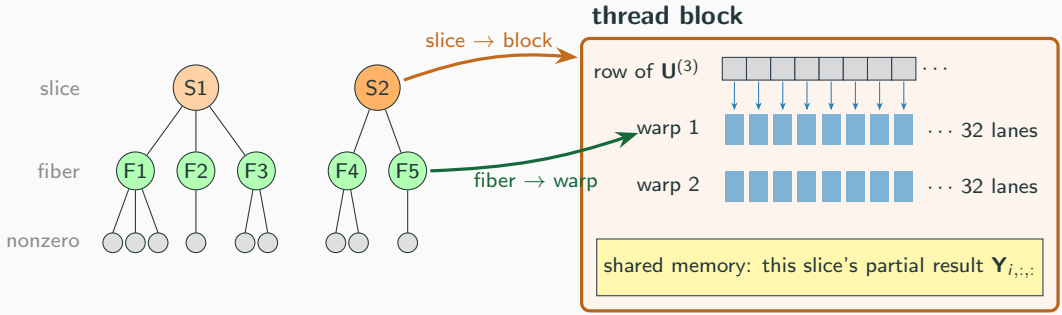
CSF keeps each fiber's nonzeros together, so T is filled and drained in one go.

Idea 2: map the CSF tree onto the GPU dynamically



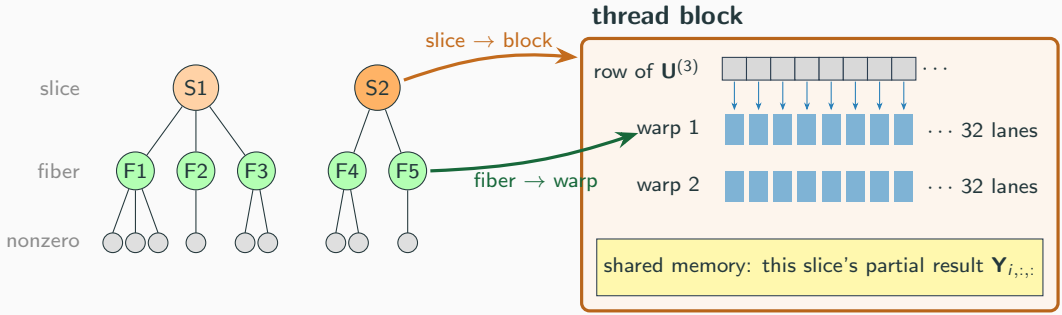
- The R_3 numbers of a factor-matrix row are spread over the 32 lanes of a warp

Idea 2: map the CSF tree onto the GPU dynamically



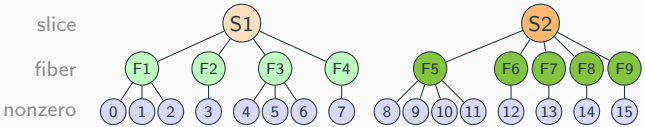
- The R_3 numbers of a factor-matrix row are spread over the 32 lanes of a warp
- Lanes read **neighbouring** numbers: coalesced memory access

Idea 2: map the CSF tree onto the GPU dynamically



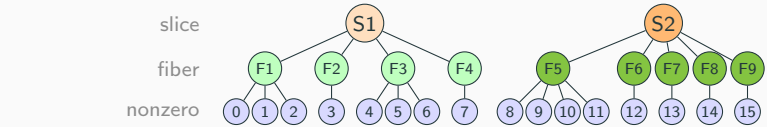
- The R_3 numbers of a factor-matrix row are spread over the 32 lanes of a warp
- Lanes read **neighbouring** numbers: coalesced memory access
- Each block sums its slice in **shared memory**: no global atomics

IntraDyn: a free warp takes the next fiber



Toy GPU: 2 SMs with one thread block (TB) each, 2 warps (W) per block, 1 nonzero per cycle.

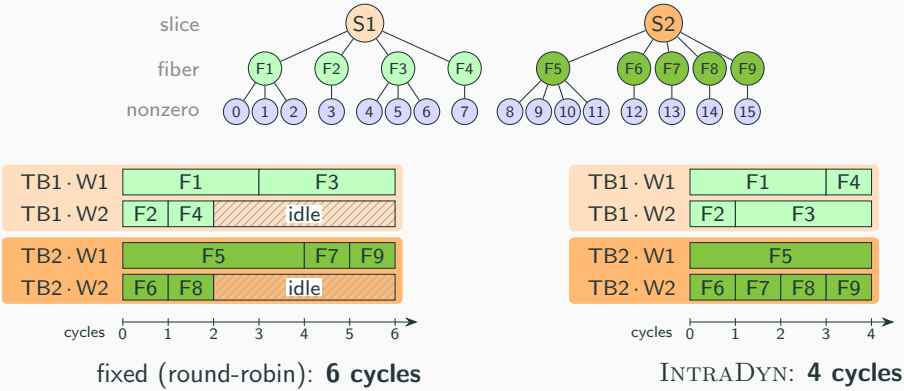
IntraDyn: a free warp takes the next fiber



fixed (round-robin): **6 cycles**

Toy GPU: 2 SMs with one thread block (TB) each, 2 warps (W) per block, 1 nonzero per cycle.

IntraDyn: a free warp takes the next fiber

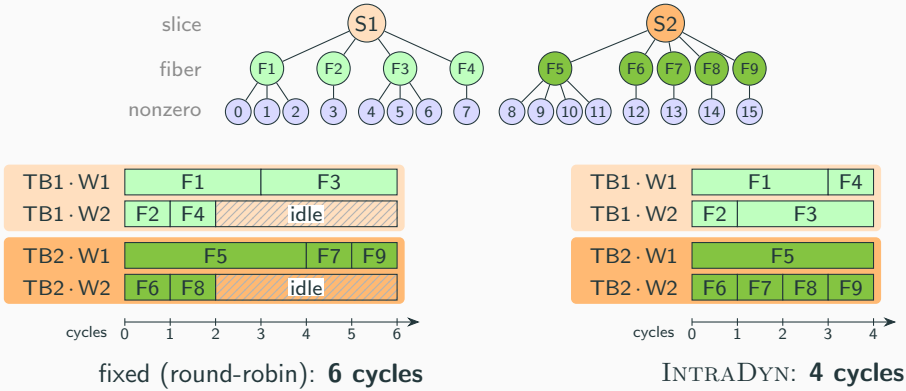


fixed (round-robin): **6 cycles**

IntraDyn: **4 cycles**

Toy GPU: 2 SMs with one thread block (TB) each, 2 warps (W) per block, 1 nonzero per cycle.

IntraDyn: a free warp takes the next fiber



fixed (round-robin): **6 cycles**

IntraDyn: **4 cycles**

Toy GPU: 2 SMs with one thread block (TB) each, 2 warps (W) per block, 1 nonzero per cycle.

Warps of a block share a counter in fast memory: whoever is free takes the next fiber. The slice's result is summed in shared memory, so there are **no global atomics**.

IntraDyn: the algorithm

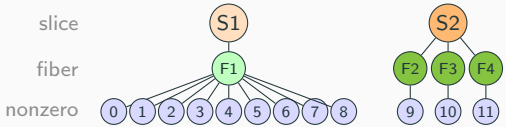
Algorithm 2 IntraDyn: **Intra**-SM **Dyn**amic Scheduling for TTMc

Require: CSF tensor $\mathcal{X}$:{ `m_idx[][]`, `m_ptr[][]`, `val[]`}, slice mode index d_s, fiber mode index d_f, nonzero mode index d_n, factor matrices $\mathbf{U}^{(2)} \in \mathbb{R}^{I_2 \times R_2}$, $\mathbf{U}^{(3)} \in \mathbb{R}^{I_3 \times R_3}$

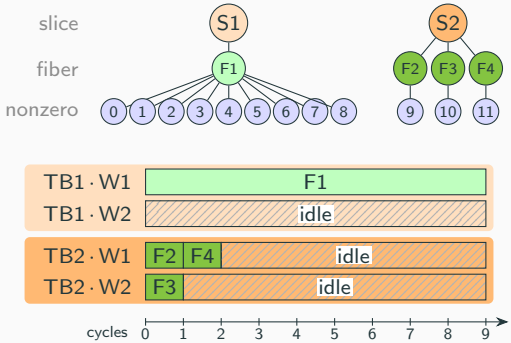
Ensure: $\mathcal{Y} \in \mathbb{R}^{I_1 \times R_2 \times R_3}$ where $y_{i\, r_2\, r_3} = \sum_{j,k} x_{ijk}\, U^{(2)}_{j\, r_2}\, U^{(3)}_{k\, r_3}$

1: **for** $\texttt{ind}_1 = 0$ to $|\texttt{m_idx}[d_s]| - 1$ **do** ▷ one thread block per slice
2: $i \leftarrow \texttt{m_idx}[d_s][\texttt{ind}_1]$
3: Allocate shared-memory buffer $\texttt{buf1} \in \mathbb{R}^{R_2 \times R_3} \leftarrow \mathbf{0}$
4: $c \leftarrow 0$ ▷ shared atomic counter for dynamic scheduling
5: **while true do** ▷ each warp dynamically claims a fiber
6: Warp leader: $\texttt{offset} \leftarrow \texttt{atomicAdd}(c, 1)$; broadcast `offset` within warp
7: $\texttt{ind}_2 \leftarrow \texttt{m_ptr}[d_f][\texttt{ind}_1] + \texttt{offset}$
8: **if** $\texttt{ind}_2 \geq \texttt{m_ptr}[d_f][\texttt{ind}_1 + 1]$ **then**
9: **break** ▷ all fibers in this slice have been claimed
10: $j \leftarrow \texttt{m_idx}[d_f][\texttt{ind}_2]$
11: FillDrain(j, $\texttt{m_ptr}[d_n][\texttt{ind}_2]$, $\texttt{m_ptr}[d_n][\texttt{ind}_2{+}1]$, `buf1`)
▷ fill and drain the fiber (factorize-and-fuse), drain uses **shared-mem** atomics
12: `__syncthreads()`
13: $\mathcal{Y}[i, :, :] \mathrel{+}= \texttt{buf1}$ ▷ write-back without atomics

FullDyn: cut heavy work into equal tasks

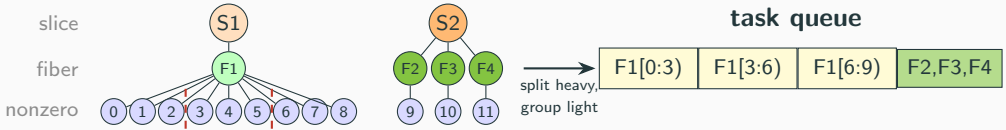


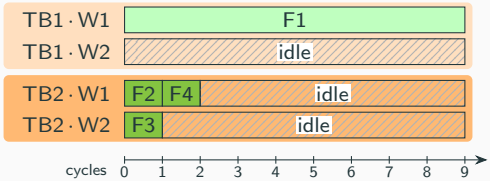
FullDyn: cut heavy work into equal tasks



fixed or IntraDyn: **9 cycles**

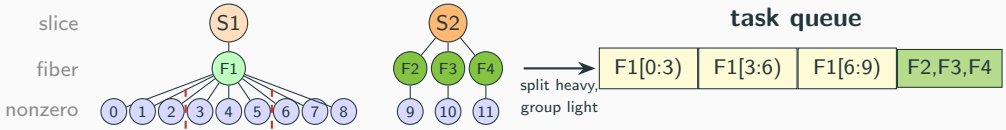
FullDyn: cut heavy work into equal tasks

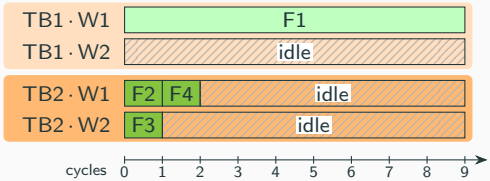




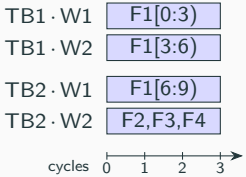
fixed or IntraDyn: **9 cycles**

FullDyn: cut heavy work into equal tasks



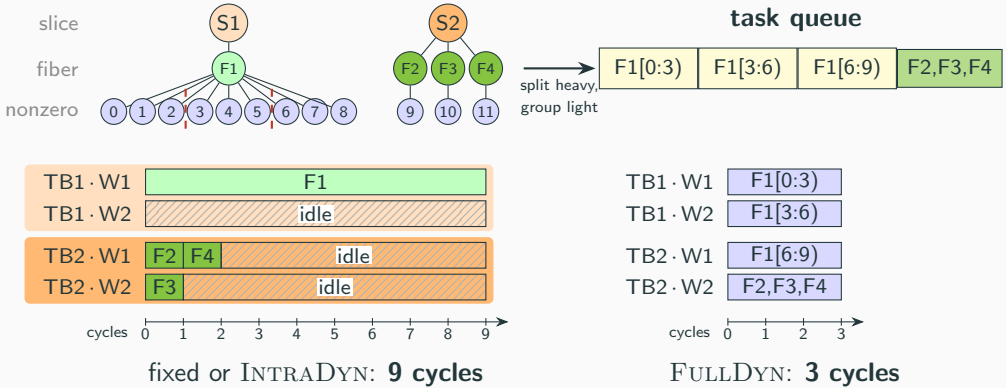


fixed or IntraDyn: **9 cycles**



FullDyn: **3 cycles**

FullDyn: cut heavy work into equal tasks



fixed or IntraDyn: **9 cycles**

FullDyn: **3 cycles**

Real case: `vast3d` has a mode with only 2 slices. IntraDyn keeps only 2 of 142 SMs busy, FullDyn is **132× faster** on that mode.

The price: a global task counter and global atomics when writing the output.

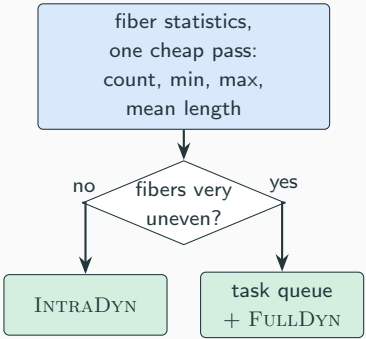
FullDyn: the algorithm

Algorithm 3 FullDyn: **Full Dyn**amic Scheduling (Intra-SM + Inter-SM) for TTMc

Require: CSF tensor $\mathcal{X}$:{ `m_idx[][]`, `m_ptr[][]`, `val[]` }, slice mode index d_s, fiber mode index d_f, nonzero mode index d_n, factor matrices $\mathbf{U}^{(2)} \in \mathbb{R}^{I_2 \times R_2}$, $\mathbf{U}^{(3)} \in \mathbb{R}^{I_3 \times R_3}$, taskqueue $\mathcal{Q}$ (from task decomposition)
Ensure: $\mathcal{Y} \in \mathbb{R}^{I_1 \times R_2 \times R_3}$

1: $g \leftarrow 0$ ▷ global atomic task counter
2: **for each** warp **in parallel** across all thread blocks **do**
3: **while true do**
4: Warp leader: `task_idx` $\leftarrow$ `atomicAdd`$(g, 1)$; broadcast within warp
5: **if** `task_idx` $\geq |\mathcal{Q}|$ **then**
6: **break**
7: $(i_p, j_b, j_e, k_s, k_e) \leftarrow \mathcal{Q}$[`task_idx`]
8: $i \leftarrow$ `m_idx`$[d_s][i_p]$
9: Initialize warp-local `buf1` $\in \mathbb{R}^{R_2 \times R_3} \leftarrow \mathbf{0}$
10: **if** $k_s < k_e$ **then** ▷ heavy-fiber tile: single fiber, tiled nonzero range
11: $j \leftarrow$ `m_idx`$[d_f][j_b]$
12: FillDrain$(j, k_s, k_e,$ `buf1`$)$ ▷ fill and drain
13: **else** ▷ light-fiber batch: full nonzero range per fiber
14: **for** `ind`$_2 = j_b$ to $j_e - 1$ **do**
15: $j \leftarrow$ `m_idx`$[d_f]$[`ind`$_2$]
16: FillDrain$(j,$ `m_ptr`$[d_n]$[`ind`$_2$], `m_ptr`$[d_n]$[`ind`$_2$+1], `buf1`$)$ ▷ fill and drain
17: `atomicAdd`$\big(\mathcal{Y}[i, :, :],$ `buf1`$\big)$ ▷ global atomic write-back

Adaptive TTMc: no single kernel wins



Chosen once per tensor and mode, before the run.

`vast3d`, its 2-slice mode



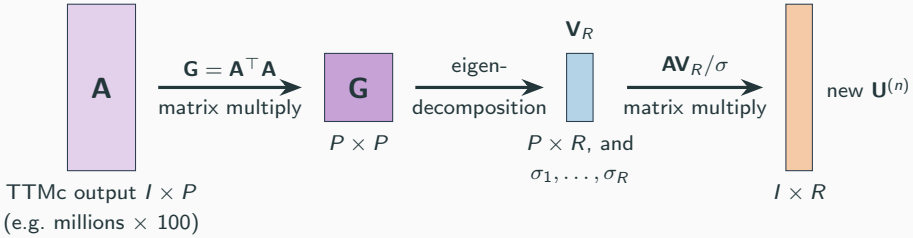
`uber`, all 4 modes



`vast3d`: 2 slices, so one block per slice leaves 140 of 142 SMs idle.

`uber`: thousands of balanced slices, so IntraDyn already balances, while FullDyn's few coarse tasks leave most warps idle.

Idea 3: shrink the SVD with the Gram matrix

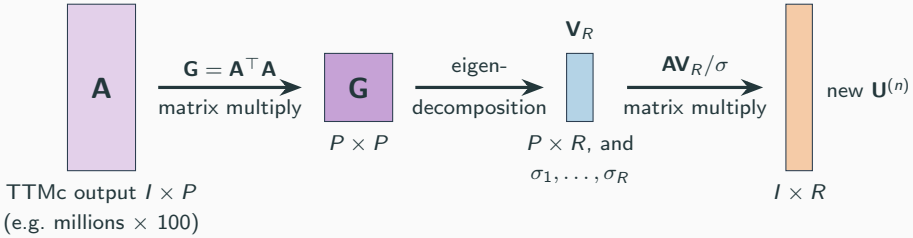


Full SVD (before)

- ✗ computes all P singular vectors
- ✗ memory grows with $I \times P$
- ✗ needs a dense, column-major copy

If **A** itself does not fit in GPU memory, **G** is built tile by tile.

Idea 3: shrink the SVD with the Gram matrix



Full SVD (before)

- ✗ computes all P singular vectors
- ✗ memory grows with $I \times P$
- ✗ needs a dense, column-major copy

Gram SVD (SAKTHI)

- ✓ only the top R vectors
- ✓ memory $\propto P^2$, independent of I
- ✓ fast matrix multiplies on the TTMc output

If **A** itself does not fit in GPU memory, **G** is built tile by tile.

Is the shortcut safe? A computable check

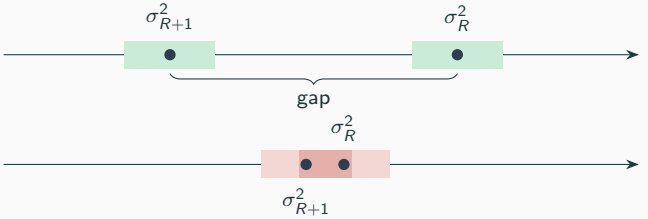
Squaring ($\mathbf{A}^\top\mathbf{A}$) magnifies rounding noise (shaded), so nearby singular values can blur together. We only need the top R, so only σ_R^2 and σ_{R+1}^2 must stay apart.



gap $>$ noise, $\mathrm{GNR} > 1$:
✔ use the fast Gram SVD

Is the shortcut safe? A computable check

Squaring ($\mathbf{A}^\top\mathbf{A}$) magnifies rounding noise (shaded), so nearby singular values can blur together. We only need the top R, so only σ_R^2 and σ_{R+1}^2 must stay apart.

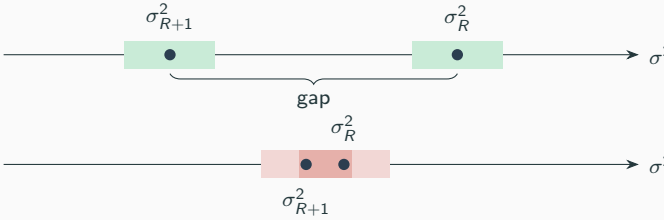


gap $>$ noise, $\mathrm{GNR} > 1$:
✔ use the fast Gram SVD

gap $<$ noise, $\mathrm{GNR} \leq 1$:
↺ fall back to the full SVD

Is the shortcut safe? A computable check

Squaring ($\mathbf{A}^\top\mathbf{A}$) magnifies rounding noise (shaded), so nearby singular values can blur together. We only need the top R, so only σ_R^2 and σ_{R+1}^2 must stay apart.



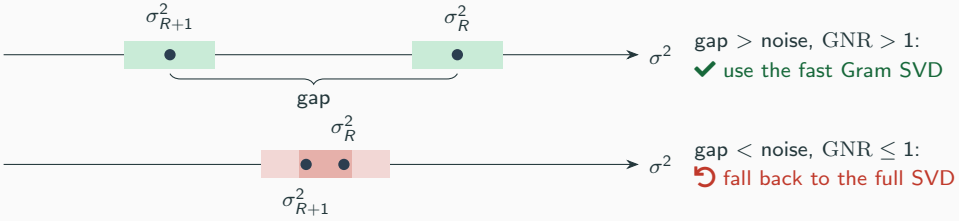
gap $>$ noise, $\mathrm{GNR} > 1$:
✔ use the fast Gram SVD

gap $<$ noise, $\mathrm{GNR} \leq 1$:
↺ fall back to the full SVD

$$\mathrm{GNR} = \frac{\sigma_R^2 - \sigma_{R+1}^2}{2c\,\epsilon_{\mathrm{mach}}\,\|\mathbf{A}\|_F^2} = \frac{\text{gap}}{\text{worst-case rounding noise}} > 1$$

Is the shortcut safe? A computable check

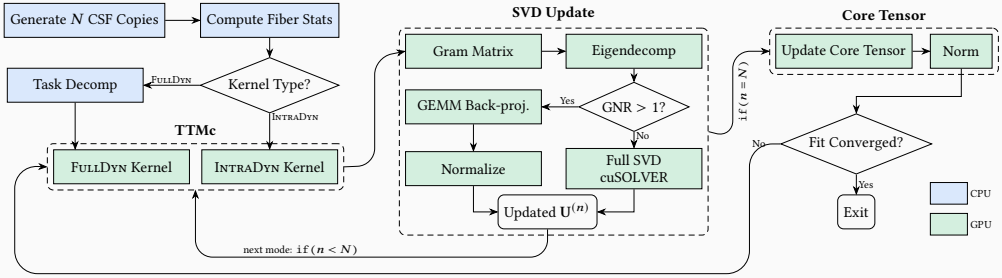
Squaring ($\mathbf{A}^\top\mathbf{A}$) magnifies rounding noise (shaded), so nearby singular values can blur together. We only need the top R, so only σ_R^2 and σ_{R+1}^2 must stay apart.



$$\mathrm{GNR} = \frac{\sigma_R^2 - \sigma_{R+1}^2}{2c\,\epsilon_{\mathrm{mach}}\,\|\mathbf{A}\|_F^2} = \frac{\text{gap}}{\text{worst-case rounding noise}} > 1$$

Do the small Gram part in double precision: ϵ_{mach} drops from 6×10^{-8} to 1.1×10^{-16}. The fast path then runs in 100% of SVD calls on 20 of 21 datasets (`nips`: 81%).

Putting it together: the SAKTHI pipeline



Results

Experimental setup

Hardware

NVIDIA L40S GPU: 142 SMs, 48 GB memory

Data

12 real tensors (network traffic, knowledge bases, social tagging, e-mail, Uber pickups, ...), up to 144M nonzeros and dimensions up to 39M
9 synthetic tensors (uniformly random)

Compared against

ParTI!: GPU HOOI, the state of the art
GPUTucker: a GPU Tucker method without TTMc and SVD

Settings

Rank 10, run until the fit converges (tolerance 10^{-5}, at most 50 iterations)

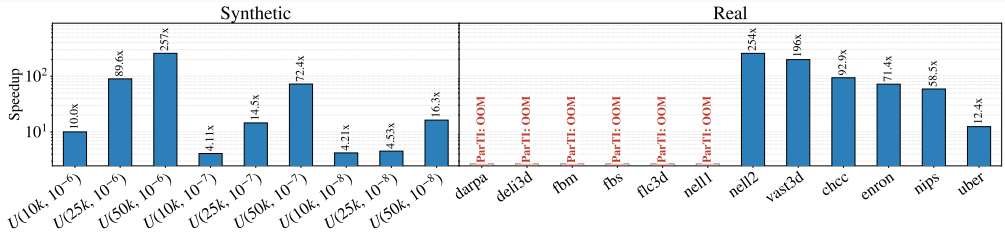
Datasets

Table 4: Real and Synthetic sparse tensor datasets used in experiments. Tensors named $U(dim, den)$ are uniformly random with equal dimensions dim, target nonzero density den, and nonzero count $u_nnz = dim^3 \times den$. ($dim \in \{10K, 25K, 50K\}$, $den \in \{10^{-6}, 10^{-7}, 10^{-8}\}$, 9 tensors total).

Name	Order	nnz	Dimensions	Density	Name	Order	nnz	Dimensions	Density
darpa	3	28M	$22K \times 22K \times 24M$	2.37×10^{-9}	nell2	3	77M	$12K \times 9K \times 29K$	2.40×10^{-5}
deli3d	3	140M	$533K \times 17M \times 2.5M$	6.14×10^{-12}	vast3d	3	26M	$165K \times 11K \times 2$	6.91×10^{-3}
fbm	3	100M	$23M \times 23M \times 166$	1.10×10^{-9}	chcc	4	5.3M	$6.2K \times 24 \times 77 \times 32$	1.46×10^{-2}
fbs	3	140M	$39M \times 39M \times 532$	1.73×10^{-10}	enron	4	54M	$6.1K \times 5.7K \times 244K \times 1.2K$	5.46×10^{-9}
flc3d	3	113M	$320K \times 28M \times 1.6M$	7.80×10^{-12}	nips	4	3.1M	$2.5K \times 2.9K \times 14K \times 17$	1.83×10^{-6}
nell1	3	144M	$2.9M \times 2.1M \times 25M$	9.05×10^{-13}	uber	4	3.3M	$183 \times 24 \times 1.1K \times 1.7K$	3.85×10^{-4}
$U(dim, den)$	3	u_nnz	$dim \times dim \times dim$	den					

Real tensors from FROSTT, HaTen2 and other public sources, synthetic tensors from GenTensor (see References).

End to end: SAKTHI vs. ParTI!



Speedup of the whole Tucker decomposition until convergence (log scale, rank 10, one-time preprocessing excluded).

33.5×
faster on average (geometric mean)

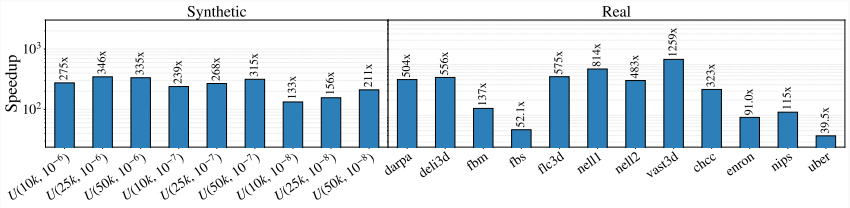
257×
best case

6 of 12
real tensors where ParTI! runs out of memory. SAKTHI finishes all.

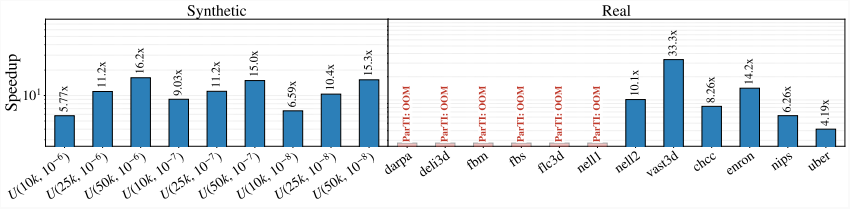
Geometric means. With one-time preprocessing included: 7.3×. Per iteration vs. GPUTucker: 17.4×.

Where the speedup comes from

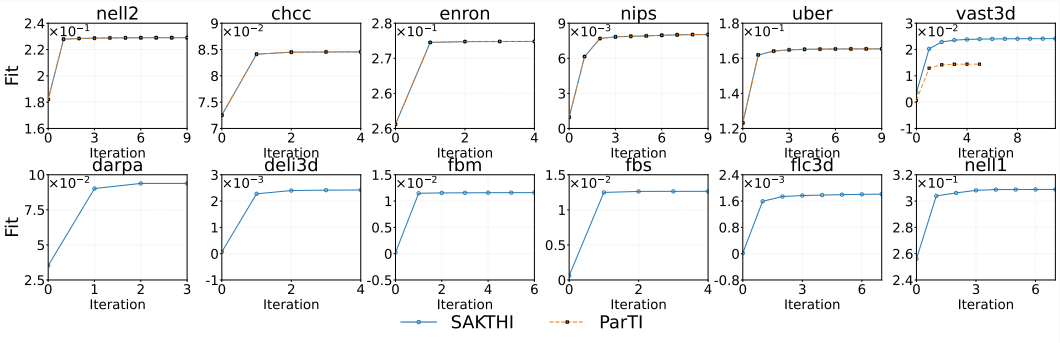
TTMc: **249**× faster (geometric mean)



SVD: **10.4**× faster (geometric mean), and it runs where the full SVD cannot



Same quality, much less time



Fit (higher is better) over the iterations on the 12 real tensors.

- Both finish on 6 tensors: the curves overlap on 5, and SAKTHI reaches a higher fit on `vast3d`
- On the other 6 tensors only SAKTHI finishes

Takeaways

For sparse tensors the SVD is **not** a side step. Optimize the **whole** HOOI pipeline.

Pick the right unit of work (the **fiber**), hand it out **dynamically**, and let each tensor's imbalance choose the kernel.

A cheap Gram SVD is safe when a **computable gap-to-noise check** passes, with the full SVD as a fallback.

SAKTHI is **33.5**× faster end to end than ParTI!, runs where ParTI! runs out of memory, and matches or beats its fit.

Thank you!

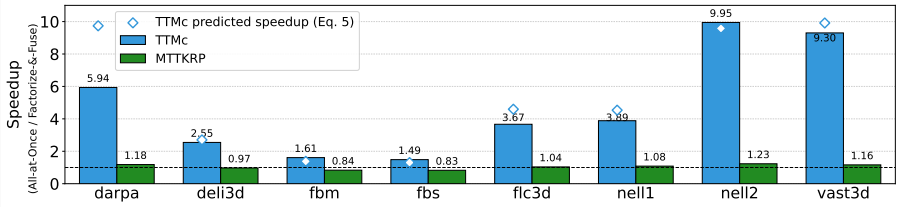
Questions?

Backup: why factorize-and-fuse is asymptotically faster

For mode-1 TTMc, $y_{ir_2r_3} = \sum_j \sum_k x_{ijk}\, U^{(2)}_{jr_2}\, U^{(3)}_{kr_3}$, with F nonzero fibers and mean fiber length $\alpha = nnz/F$:

$$C_{\text{all-at-once}} = nnz \cdot R_2 R_3, \qquad C_{\text{factorize-and-fuse}} = R_3\,(nnz + F \cdot R_2)$$

$$\frac{C_{\text{all-at-once}}}{C_{\text{factorize-and-fuse}}} = \frac{\alpha\, R_2}{\alpha + R_2} \leq \min(\alpha, R_2)$$



Measured on CPU with all ranks 10. Diamonds: predicted TTMc speedup. For MTTKRP (CP decomposition) the same trick gives no asymptotic gain.

Backup: the resolution condition

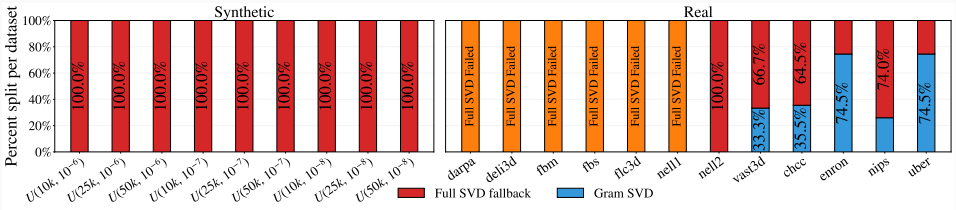
Theorem. Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ ($m \geq n$) have singular values $\sigma_1 \geq \cdots \geq \sigma_n \geq 0$, let ϵ_{mach} be the unit roundoff, and let $c = c_1 + c_2$ with $c_1 = O(m)$ from forming the Gram matrix and $c_2 = O(n)$ from the symmetric eigensolver. The eigenvalue method on $\mathbf{A}^\top \mathbf{A}$ can reliably separate $\sigma_i > \sigma_j$ only if

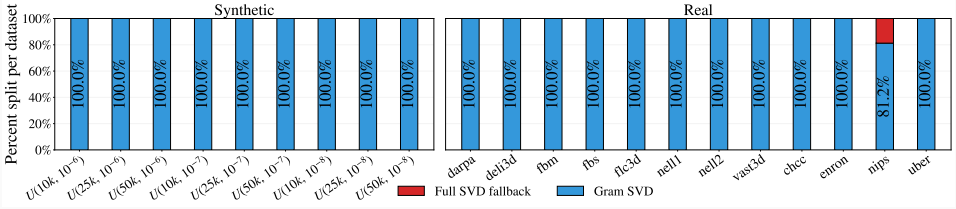
$$\mathrm{GNR}_{ij} = \frac{\sigma_i^2 - \sigma_j^2}{2c\,\epsilon_{\mathrm{mach}}\,\|\mathbf{A}\|_F^2} > 1.$$

Proof idea. The computed eigenvalues are exact for $\mathbf{A}^\top \mathbf{A} + \mathbf{E}_1 + \mathbf{E}_2$, where $\|\mathbf{E}_1 + \mathbf{E}_2\|_2 \leq c\,\epsilon_{\mathrm{mach}} \|\mathbf{A}\|_F^2$. Weyl's theorem bounds how far each eigenvalue can move. In the worst case the gap shrinks by twice that bound.

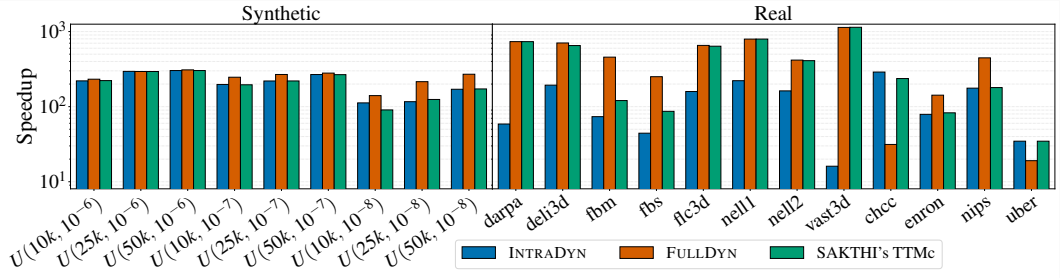
SAKTHI checks $\mathrm{GNR}_{R_n, R_n+1} > 1$ at run time and falls back to the full SVD otherwise.

Backup: how often the fast path is taken





Backup: adaptive TTMc on every dataset



TTMc speedup over ParTI! of IntraDyn, FullDyn and SAKTHI's adaptive choice (green).

Backup: GPU counters for IntraDyn and FullDyn

	`vast3d` (2-slice mode)		`uber` (4-mode sweep)	
Metric	IntraDyn	FullDyn	IntraDyn	FullDyn
Achieved occupancy (%)	66.7	64.6	91.8 to 97.7	2.9 to 4.2
SM active cycles, min/mean	0.00	0.99	0.91 to 0.99	0.00 to 0.03
SM active cycles, max/mean	139.7	1.19	1.00 to 1.09	1.8 to 5.0
Global atomics (task claims)	0	65.2K	0	39.5K
Global reductions (output)	0	560.7K	1.76M	1.76M
Sectors per global load request	1.33	1.33	1.16 to 1.17	1.31
TTMc time (ms)	195.4	1.48	1.28	39.2

NVIDIA Nsight Compute, $R = 10$, second HOOI iteration. Times come from unprofiled runs.

References i

- Ahmad, Khalid et al. (Feb. 2024). **"Exploring Data Layout for Sparse Tensor Times Dense Matrix on GPUs".** In: *ACM Transactions on Architecture and Code Optimization* 21.1, 20. DOI: 10.1145/3633462.
- Burch, Myson et al. (2025). **Towards Quantum Tensor Decomposition in Biomedical Applications.** DOI: 10.48550/arXiv.2502.13140. arXiv: 2502.13140.
- Carlson, Andrew et al. (2010). **"Toward an Architecture for Never-Ending Language Learning.".** In: *AAAI*. Vol. 5, p. 3.
- Globerson, A. et al. (2007). **"Euclidean Embedding of Co-occurrence Data".** In: *The Journal of Machine Learning Research* 8, pp. 2265–2295.
- Görlitz, Olaf, Sergej Sizov, and Steffen Staab (2008). **"PINTS: peer-to-peer infrastructure for tagging systems.".** In: *IPTPS*, p. 19.

References ii

- Jeon, Inah et al. (2015). **"HaTen2: Billion-scale Tensor Decompositions".** In: *IEEE International Conference on Data Engineering (ICDE)*.
- Kolda, Tamara G. and Brett W. Bader (Aug. 2009). **"Tensor Decompositions and Applications".** en. In: *SIAM Review* 51.3, pp. 455–500. ISSN: 0036-1445, 1095-7200. DOI: 10.1137/07070111x. URL: https://epubs.siam.org/doi/10.1137/07070111X (visited on 07/24/2025).
- Liu, Bangtian et al. (2017). **"A Unified Optimization Approach for Sparse Tensor Operations on GPUs".** In: *2017 IEEE International Conference on Cluster Computing (CLUSTER)*, pp. 47–57. DOI: 10.1109/CLUSTER.2017.75.

References iii

- Lu, Haiping, Konstantinos N. Plataniotis, and Anastasios N. Venetsanopoulos (Jan. 2008). **"MPCA: Multilinear Principal Component Analysis of Tensor Objects".** In: *IEEE Transactions on Neural Networks* 19.1, pp. 18–39. DOI: 10.1109/TNN.2007.901277.
- Ma, Yuchen et al. (July 2019). **"Optimizing sparse tensor times matrix on GPUs".** en. In: *Journal of Parallel and Distributed Computing* 129, pp. 99–109. ISSN: 07437315. DOI: 10.1016/j.jpdc.2018.07.018. URL: https://linkinghub.elsevier.com/retrieve/pii/S0743731518305161 (visited on 07/11/2024).
- Shetty, Jitesh and Jafar Adibi (2004). **"The Enron email dataset database schema and brief statistical report".** In: *Information sciences institute technical report, University of Southern California* 4.

References iv

- Sidiropoulos, Nicholas D et al. (2017). **"Tensor decomposition for signal processing and machine learning".** In: *IEEE Transactions on signal processing* 65.13, pp. 3551–3582.
- Smith, Shaden et al. (2017). **FROSTT: The Formidable Repository of Open Sparse Tensors and Tools.** URL: http://frostt.io/.
- Torun, Tugba, Ameer Taweel, and Didem Unat (Mar. 2025). **A Sparse Tensor Generator with Efficient Feature Extraction.** en. arXiv:2405.04944 [cs]. DOI: 10.48550/arXiv.2405.04944. URL: http://arxiv.org/abs/2405.04944 (visited on 06/18/2025).
- Wang, Haotian et al. (June 2023a). **"A Heterogeneous Parallel Computing Approach Optimizing SpTTM on CPU-GPU via GCN".** In: *ACM Transactions on Parallel Computing* 10.2, 9. DOI: 10.1145/3584373.

- Wang, Haotian et al. (Aug. 2023b). **"A Novel Parallel Algorithm for Sparse Tensor Matrix Chain Multiplication via TCU-Acceleration".** In: *IEEE Transactions on Parallel and Distributed Systems* 34.8, pp. 2419–2432. ISSN: 1558-2183. DOI: 10.1109/TPDS.2023.3288520. URL: https://ieeexplore.ieee.org/document/10159508/ (visited on 01/29/2026).
- Whiting, Mark et al. (2015). **"VAST Challenge 2015: Mayhem at dinofun world".** In: *Visual Analytics Science and Technology (VAST), 2015 IEEE Conference on*. IEEE, pp. 113–118.
- Yin, Miao et al. (2022). **"BATUDE: Budget-Aware Neural Network Compression Based on Tucker Decomposition".** In: *Proceedings of the AAAI Conference on Artificial Intelligence* 36.8, pp. 8874–8882. DOI: 10.1609/aaai.v36i8.20869.